Escola de Engenharia
Programa de Pós-Graduação em Engenharia Elétrica

EEE889 - Eletromagnetismo Computacional

Raphael Henrique Braga Leivas

LISTA 3

Belo Horizonte
26 de abril de 2026

SUMÁRIO

1 EXERCÍCIO 1 3
2 EXERCÍCIO 2 4
2.1 Solução 4
2.2 Medida de erro 4
3 EXERCÍCIO 3 7
3.1 Excitação gaussiana 7
3.2 Excitação senoidal 8
4 EXERCÍCIO 4 12
4.1 Excitação gaussiana 12
4.2 Excitação senoidal 13
5 EXERCÍCIO 4.3 (LIVRO) 16
5.1 Item (a) 17
5.2 Item (b) 18
6 EXERCÍCIO 4.4 (LIVRO) 20
6.1 Item (a) 20
6.2 Item (b) 21
6.3 Item (c) 21
7 EXERCÍCIO 4.6 (LIVRO) 25
7.1 Item (a) 25

ANEXO A CÓDIGO EXERCÍCIO 2 28
ANEXO B CÓDIGO EXERCÍCIO 3 30
ANEXO C CÓDIGO EXERCÍCIO 4 32
ANEXO D CÓDIGO EXERCÍCIO 4.3 35
ANEXO E CÓDIGO EXERCÍCIO 4.4 - MODO TM 37
ANEXO F CÓDIGO EXERCÍCIO 4.4 - MODO TE 38
ANEXO G CÓDIGO EXERCÍCIO 4.6 40

1 EXERCÍCIO 1

Vamos discretizar uma das equações de Maxwell em 1 dimensão no vácuo, como por exemplo

$$\frac{\partial E_y}{\partial t} = -\frac{1}{\epsilon}\frac{\partial H_z}{\partial z} \tag{1.1}$$

Usando diferenças centradas, cada uma das derivadas de (1.1) pode ser aproximada por

$$\frac{\partial E_y}{\partial t} = \frac{E^{n+1} - E^n}{\Delta t} + O\left(\Delta t^2\right) \quad , \quad \frac{\partial H_z}{\partial z} = \frac{H_{i+1/2} - H_{i-1/2}}{\Delta x} + O\left(\Delta x^2\right)$$

onde as diferenças centradas resultam em erros de ordem 2. O erro τ_E é dado pela diferença entre os lados esquerdo e direito da Equação (1.1) após substituir as aproximações, ou seja,

$$\begin{aligned}
\tau_E &= \frac{E^{n+1} - E^n}{\Delta t} + O\left(\Delta t^2\right) - \frac{1}{\epsilon}\left(\frac{H_{i+1/2} - H_{i-1/2}}{\Delta x} + O\left(\Delta x^2\right)\right) \\
&= \left[\frac{E^{n+1} - E^n}{\Delta t} - \frac{1}{\epsilon}\frac{H_{i+1/2} - H_{i-1/2}}{\Delta x}\right] + O\left(\Delta t^2\right) + \frac{1}{\epsilon}O\left(\Delta x^2\right) \\
&= \left[\frac{E^{n+1} - E^n}{\Delta t} - \frac{1}{\epsilon}\frac{H_{i+1/2} - H_{i-1/2}}{\Delta x}\right] + C\Delta t^2 + \frac{C}{\epsilon}\Delta x^2
\end{aligned}$$

Aplicando a Equação 1.1 para substituir o termo entre colchetes, temos:

$$\tau_E \leq C\left(\Delta t^2 + \Delta x^2\right)$$

2 EXERCÍCIO 2

2.1 *Solução*

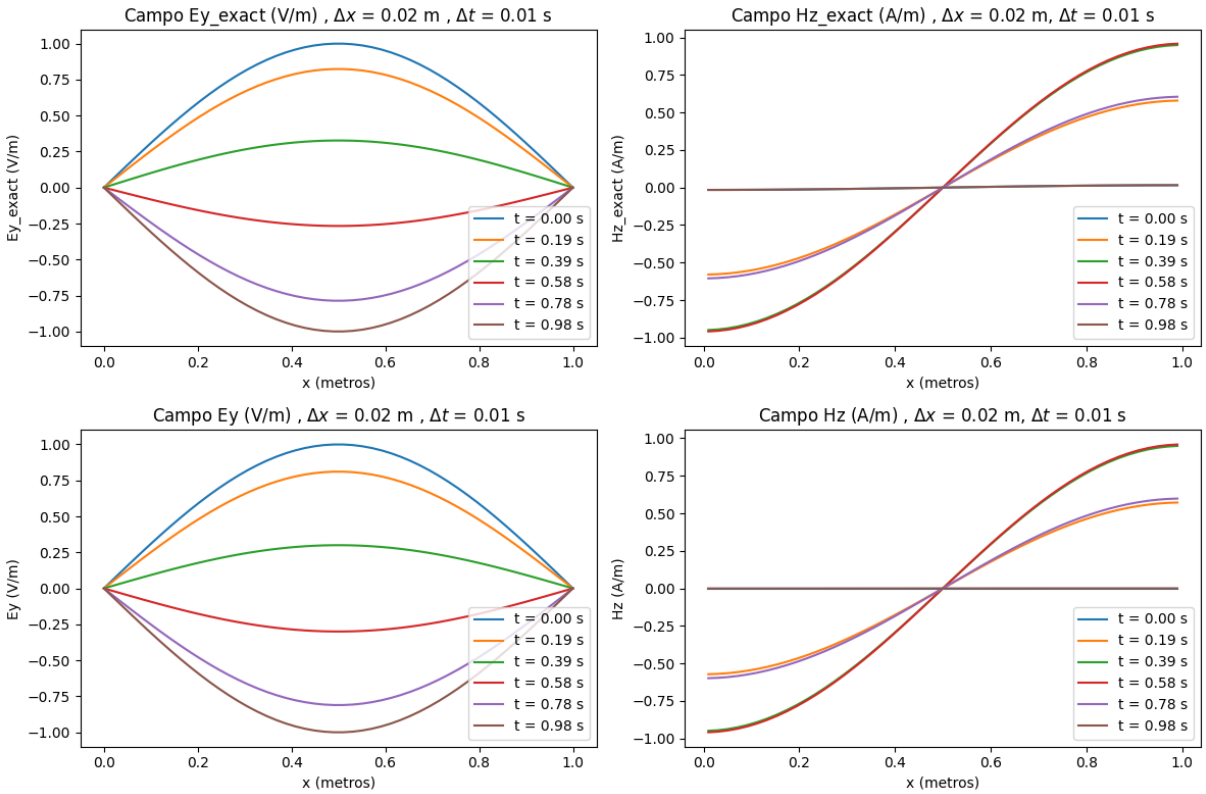
Todo o código usado nessa solução encontra-se no Anexo A.

Os seguintes parâmetros de discretização foram usados:

- $N_x = 50 \quad , \quad N_t = 100$
- $\Delta x = 0.02$ m
- $\Delta t = 0.01$ s
- $T = 1$ s

A solução numérica (E_y e H_z) obtida é semelhante com a solução analítica ($E_{y-exact}$ e $H_{z-exact}$) tanto para o campo magnético quando para o campo elétrico, conforme mostra a Figura 1.

Figura 1 – Comparação entre as soluções analítica (exact) e numéricas obtidas.

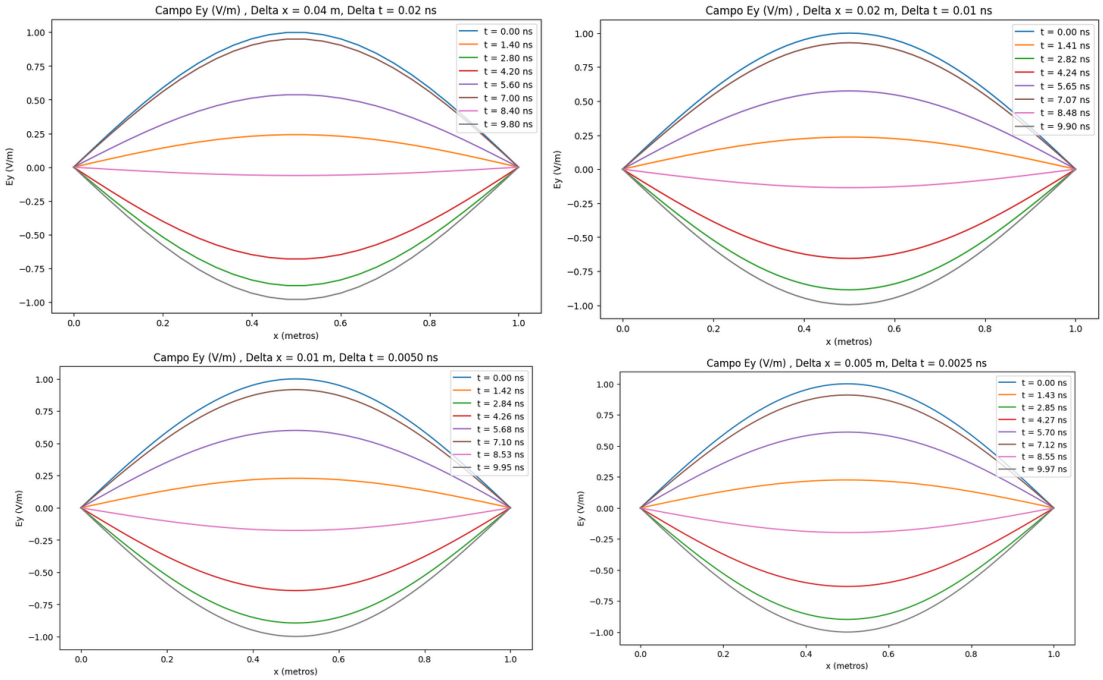


Variando os valores dos parâmeotros Δx e Δt, vemos que a solução não sofre muitas alterações, como mostra a Figura 2.

2.2 *Medida de erro*

Usando a medida de erro proposta no enunciado, vemos que o erro se reduz por um fator de $1/4$ quando reduzimos o valor de Δx pela metade, conforme esperado dado que o esquema de Yee é de ordem 2, como mostra a Tabela 1.

Figura 2 – Solução numérica do exercício 2 para diferentes discretizações.



A ordem p é obtida através de

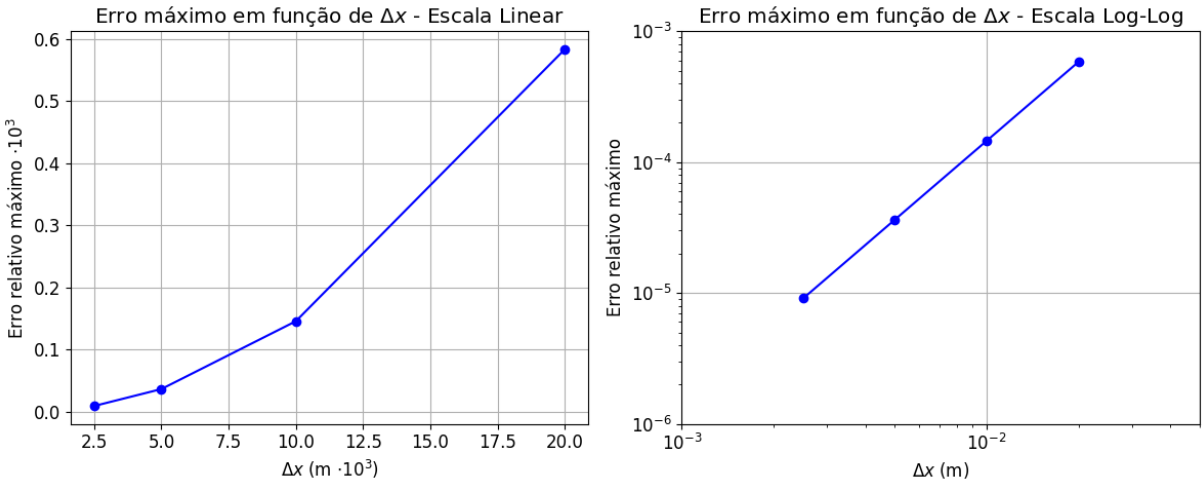
$$p = \frac{\ln\left(\frac{e_2}{e_1}\right)}{\ln\left(\frac{\Delta x_2}{\Delta x_1}\right)} = \frac{\ln\left(\frac{e_2}{e_1}\right)}{\ln 0.5}$$

Tabela 1 – Tabela de convergência do método para diferentes parâmetros de discretização.

N_x	Δt	Δx	Rel. Err. $\cdot 10^3$	Razão e_2/e_1	Ordem p
50	0.02	0.04	0.583	-	-
100	0.01	0.02	0.145	0.2495	2.002
200	0.005	0.01	0.036	0.2497	2.001
400	0.0025	0.005	0.009	0.2498	2.000

A redução do erro também está exibida graficamente na Figura 3, tanto em escala linear quanto em escala logarítimica. Note que na escala log a reta tem inclinação igual a 2, conforme esperado pela ordem.

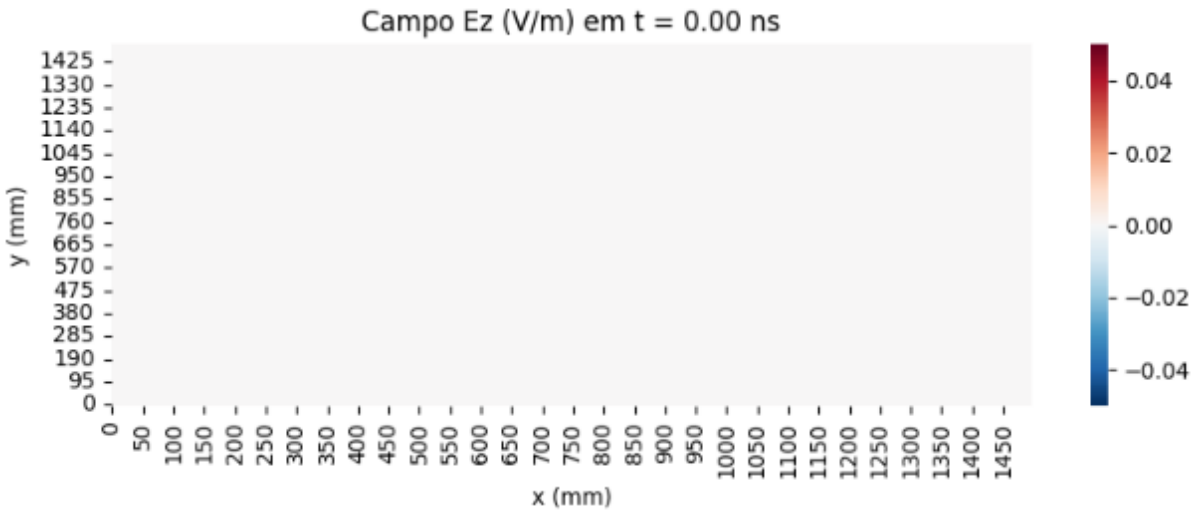
Figura 3 – Convergência do erro relativo máximo obtido.



3 EXERCÍCIO 3

O primeiro passo é montar um grid nas condições do problema, como mostra a Figura 4. Esse grid comporta uma cavidade circular de raio 0.5 m. No instante $t = 0$, não há campo elétrico e nenhuma excitação introduzida, sendo esse o estado inicial do grid.

Figura 4 – Grid para o exercício 3.



Os seguintes parâemtros de discretização foram usados:

- $N_x = N_y = 300$
- $\Delta x = \Delta y = 5$ mm
- $\Delta t = 10^{-12}$ s
- N_t = 2000

Note que esses parâmetros satisfazem a condição de estabilidade para o método de Yee em 2D, uma vez que

$$\Delta t \leq \frac{1}{c\sqrt{\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}}} \quad \Longrightarrow \quad 10^{-12} \leq 11.79 \cdot 10^{-12}$$

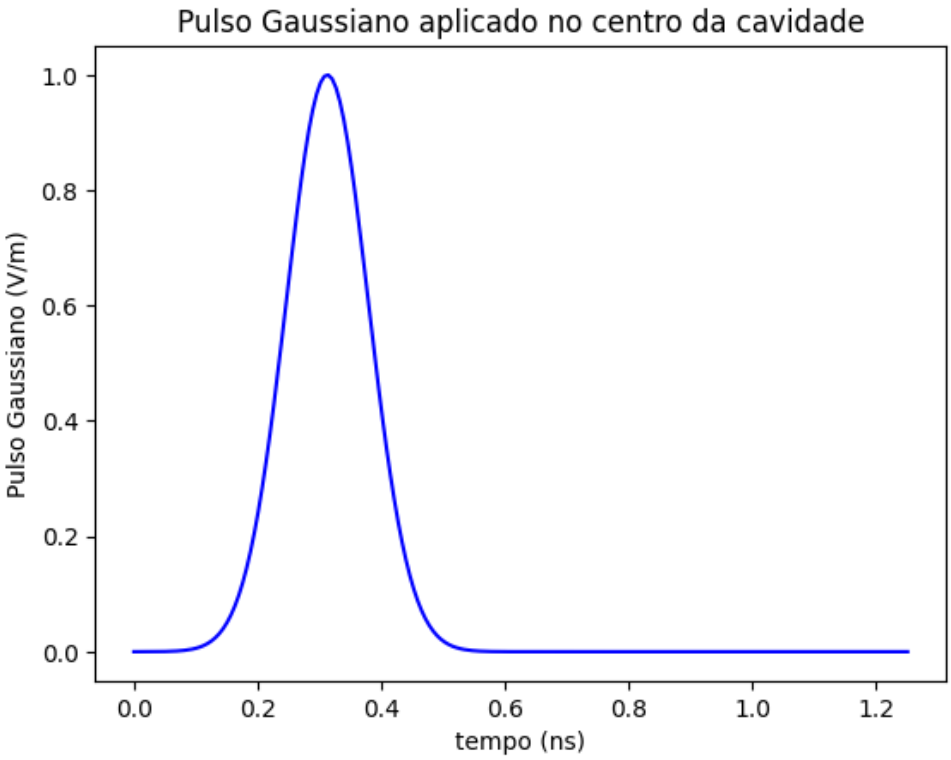
3.1 Excitação gaussiana

Todo o código usado nessa solução encontra-se no Anexo B.

Aplicamos um pulso gaussiano no centro do grid com o perfil temporal exibido na Figura 5. O pulso é dado por

$$E(t) = \exp\left(-\frac{(t - t_0)^2}{\sigma^2}\right)$$

Figura 5 – Pulso gaussiano para o exercício 3.



onde $t_0 = 50\Delta t$ e $\sigma = 15$. Assim, o pulso cresce suavamente a partir de zero e atinge seu pico em $t = t_0$, para depois decrescer novamente.

Usando o esquema de Yee em duas dimensões para o modo Transversal Magnético (TM), obtemos a seguinte evolução do campo elétrico E_z ao longo do tempo, exibida na Figura 6. Vemos que o pulso gaussiano se propaga para fora a partir do centro da cavidade.

Para simular reflexão nas paredes da cavidade, consideramos a cavidade como um condutor elétrico perfeito (PEC), o que implica que o campo elétrico tangencial deve ser zero nas paredes. Por simplicidade, também consideramos o campo elétrico zero para fora da cavidade. Assim, temos $E_z = 0$ nas bordas do grid, o que simula a reflexão do pulso gaussiano quando ele atinge as paredes da cavidade. A ressonância do campo ao longo do tempo está exibida na Figura 7.

Para calcular a frequência de ressonância, coletamos valores de E_z em um ponto fixo da cavidade e plotamos sua transformada de Fourier. Tomando o ponto central do grid, obtemos a seguinte transformada de Fourier, exibida na Figura 8.

3.2 Excitação senoidal

Aplicando uma senoide de $f = 1$ GHz e amplitude unitária no centro da cavidade, obtemos o seguinte perfil de ressonância para o campo elétrico E_z, exibido na Figura 9. A FFT do campo elétrico no centro da cavidade exibe um pico claro em $f = 1$ GHz, como mostra a Figura 10.

Figura 6 – Evolução do campo elétrico E_z para o exercício 3.

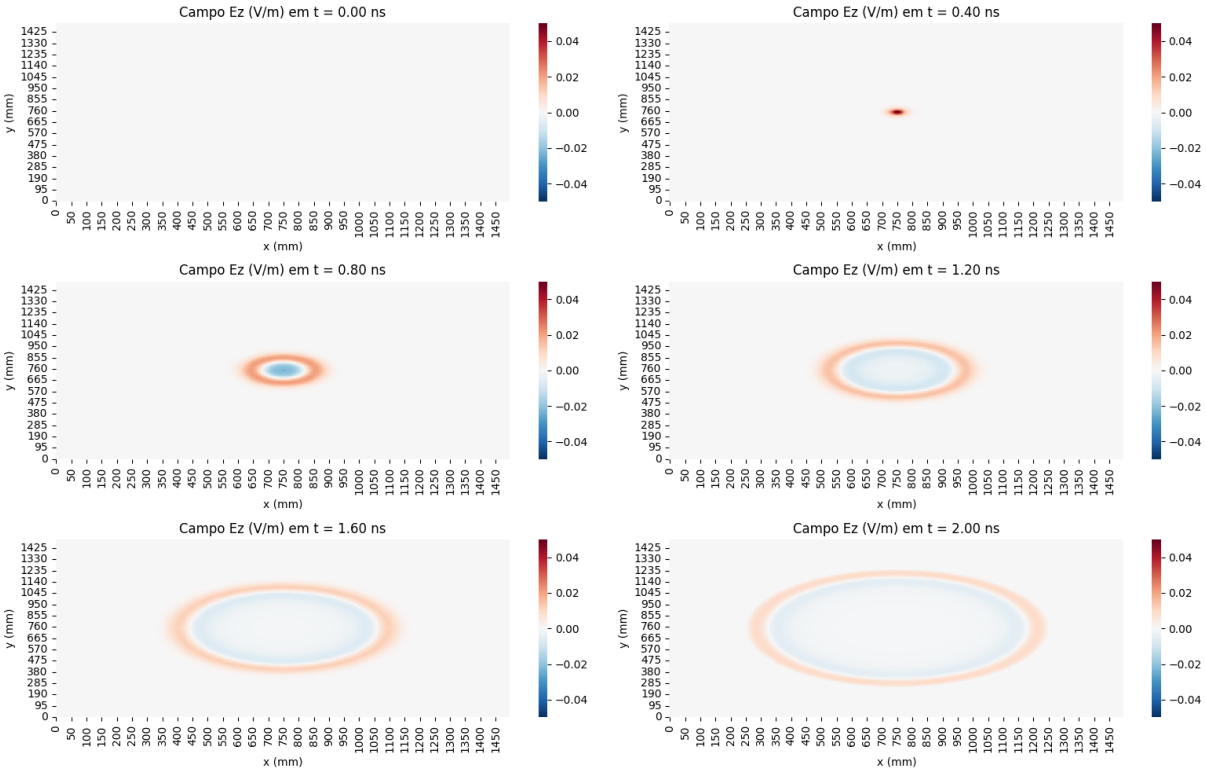


Figura 7 – Ressonância do campo elétrico E_z para o exercício 3.

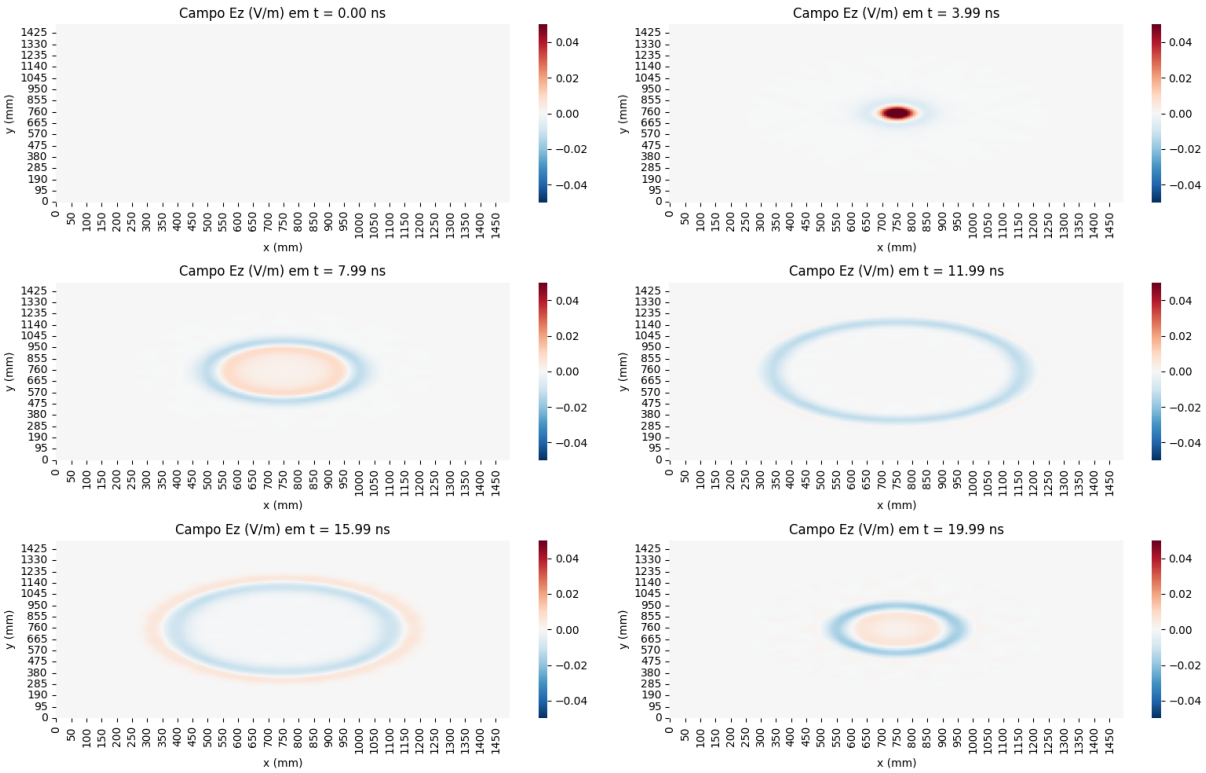


Figura 8 – Espectro FFT do campo elétrico E_z para o exercício 3.

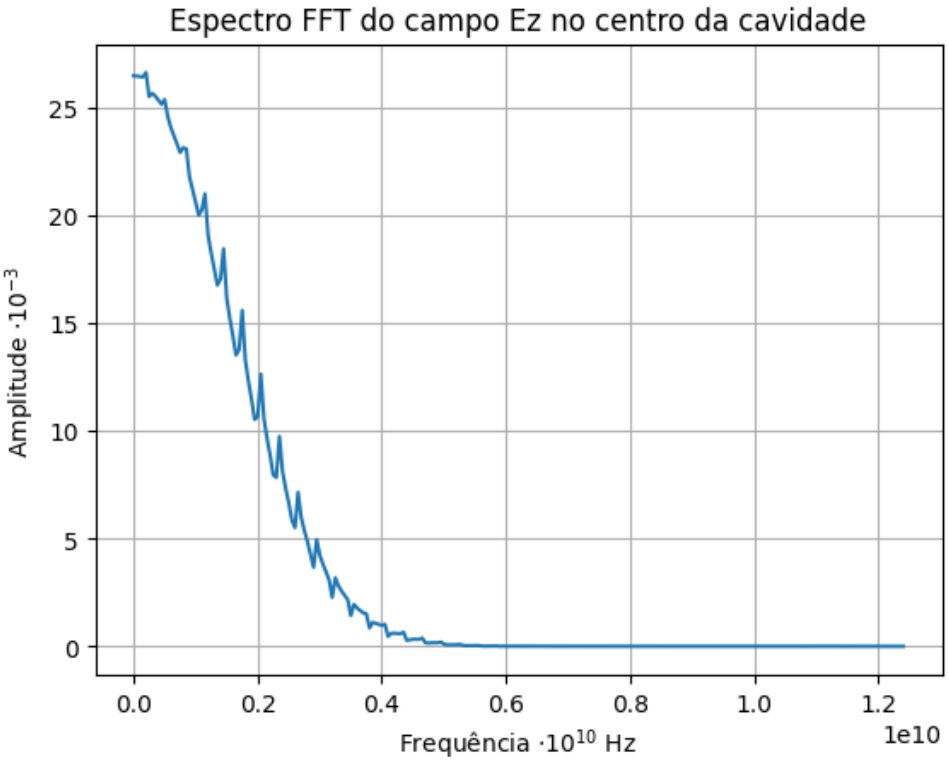


Figura 9 – Ressonância do campo elétrico E_z para o exercício 3, excitação senoidal.

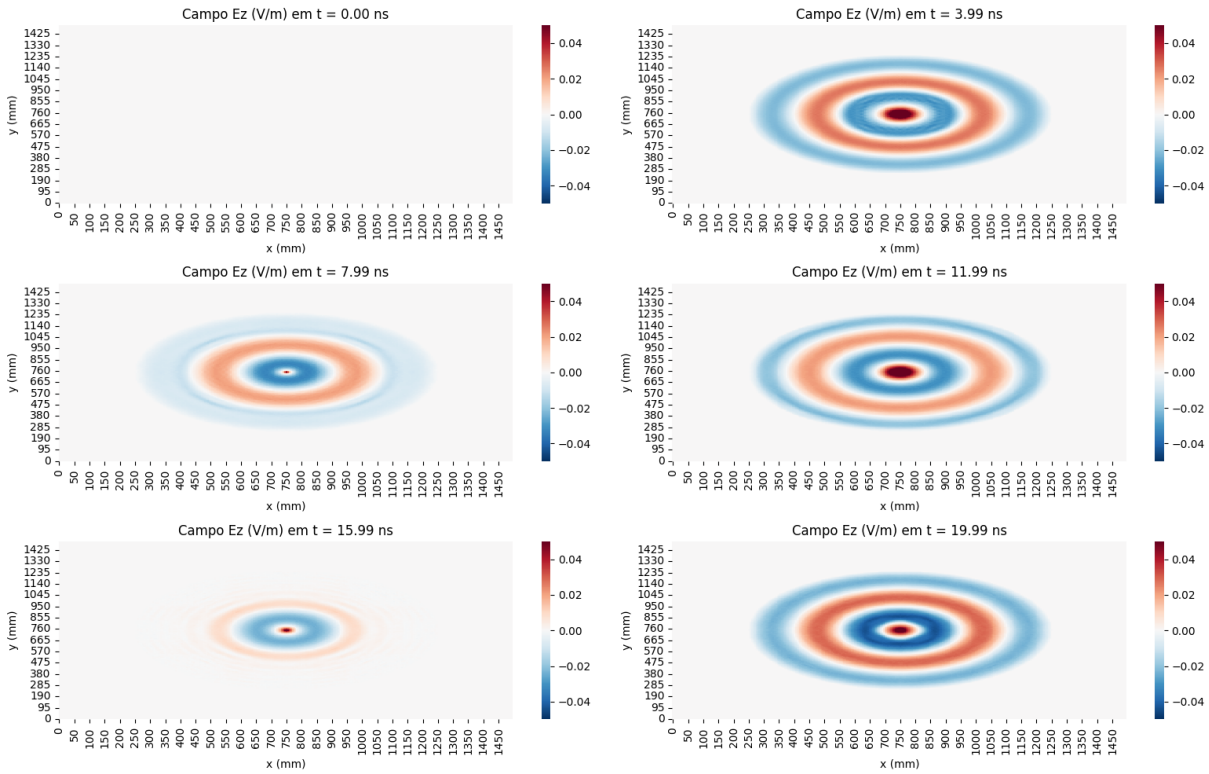
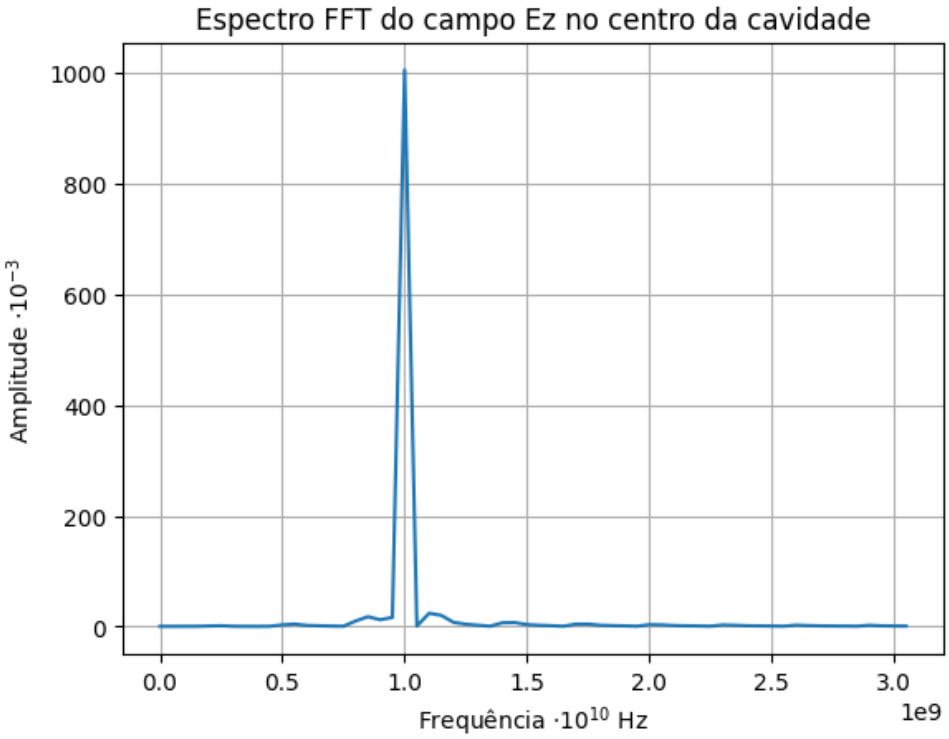


Figura 10 – Espectro FFT do campo elétrico E_z para o exercício 3, excitação senoidal.

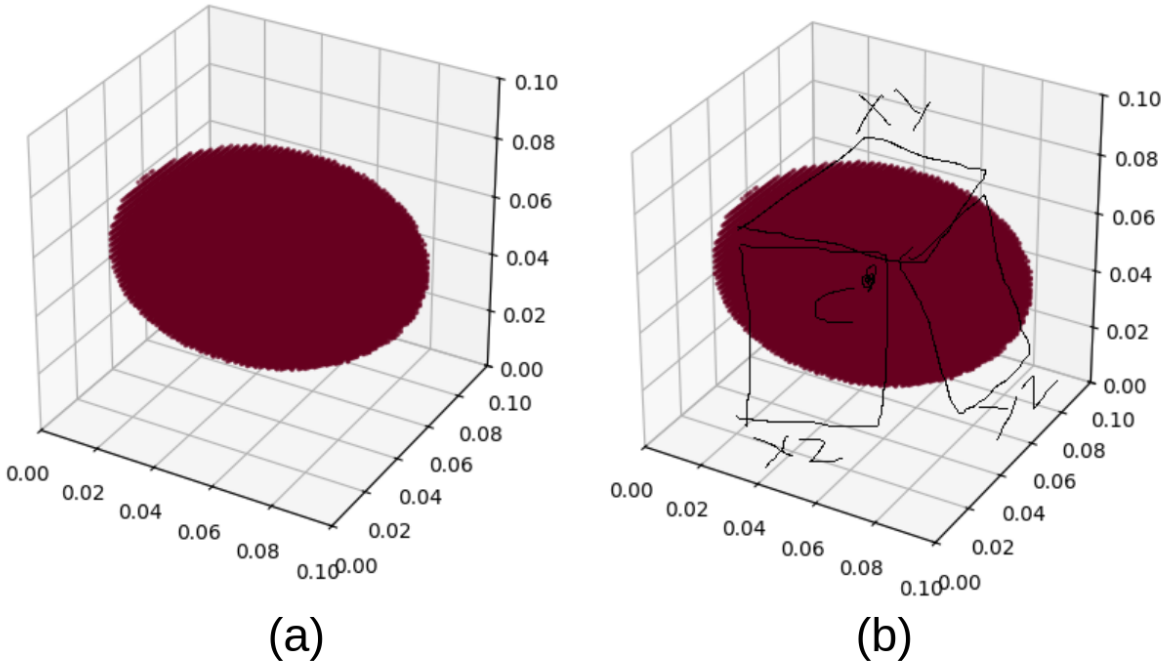


4 EXERCÍCIO 4

Todo o código desse exercício encontra-se no Anexo C.

O primeiro passo é desenhar o domínio, como mostra a Figura 11 (a). O centro da elipse é o ponto C, e os planos XY, XZ e YZ serão usados para analisar o valor dos campos em uma projeção bidimensional, facilitando a análise dos gráficos. A elipse representa o interior de uma cavidade metálica, cujas paredes são assumidas ser um condutor elétrico perfeito (condição de contorno PEC). O ponto de análise do espectro de frequência está localizado a 20 mm do centro.

Figura 11 – (a) Cavidade elíptica do problema 4 e (b) planos de análise, a uma distância de 20 mm do centro C.



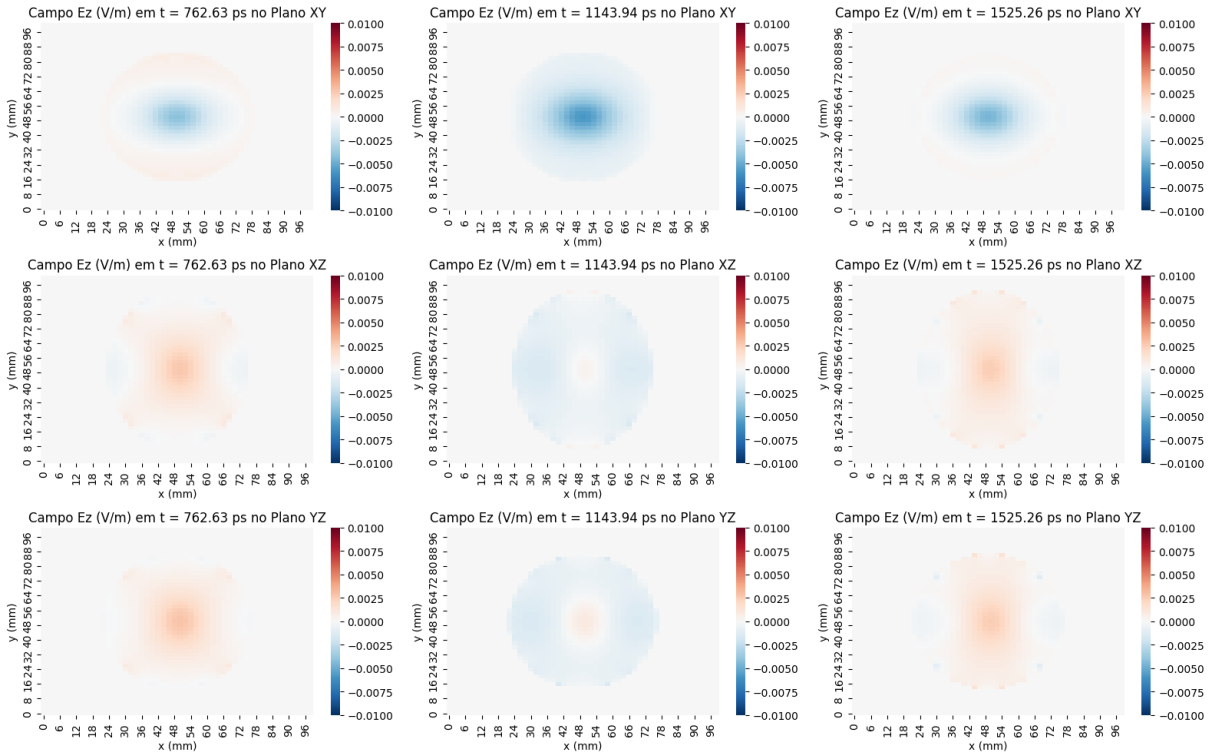
Os parâmetros de discretização usados foram:

- $N_x = N_y = N_z = 50 \quad , \quad N_t = 600$
- $\Delta x = \Delta y = \Delta z = 2$ mm
- $\Delta t = 3 \cdot 10^{-12}$ s

4.1 *Excitação gaussiana*

No centro da cavidade, aplicamos uma excitação gaussiana igual à usada no Exercício 3. A Figura 12 mostra as projeções do campo E_z em diferentes instantes de tempo nos três planos de análise. Vemos que o campo se propaga nas três direções, e o ponto de análise tem o espectro de frequência exibido na Figura 13. Usando `scatter`, é possível obter uma visualização 3D da dispersão do campo E_z, exibida na Figura **??**.

Figura 12 – Evolução do campo E_z ao longo do tempo através dos três planos de análise.



4.2 *Excitação senoidal*

No centro da cavidade, aplicamos uma excitação senoidal dada por

$$E_z(n) = \sin\left(2\pi f n \Delta t\right)$$

onde $f = 10^9$ Hz. A Figura 15 mostra as projeções do campo E_z em diferentes instantes de tempo nos três planos de análise. Vemos que o campo se propaga nas três direções, e o ponto de análise tem o espectro de frequência exibido na Figura 16. Vemos um pico na frequência de 1 GHz, conforme esperado.

Figura 13 – Transformada de Fourier do sinal coletado no ponto de análise a 20 mm do centro da cavidade.

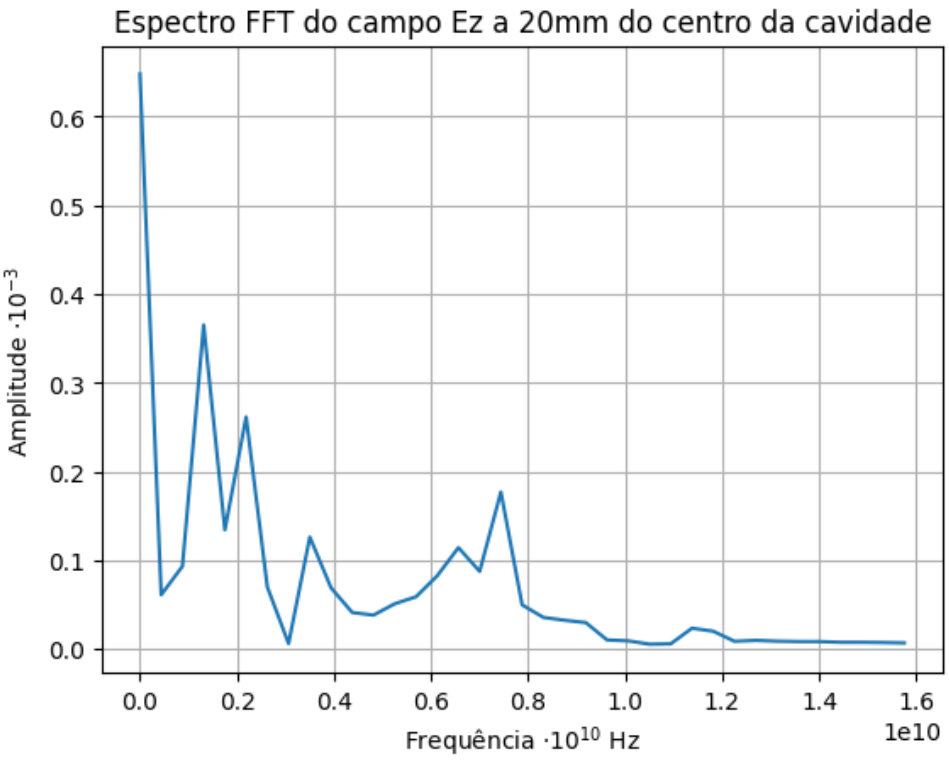


Figura 14 – Dispersão 3D do campo E_z ao longo do tempo dentro da cavidade elíptica.

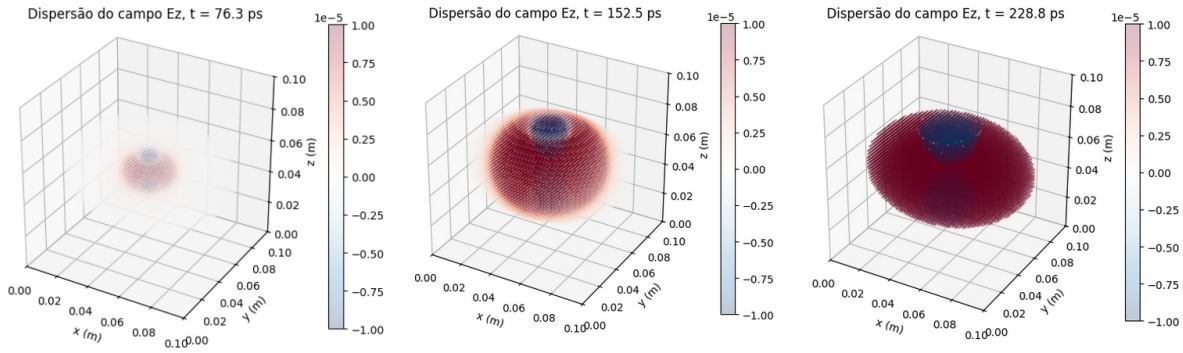


Figura 15 – Evolução do campo E_z ao longo do tempo através dos três planos de análise.

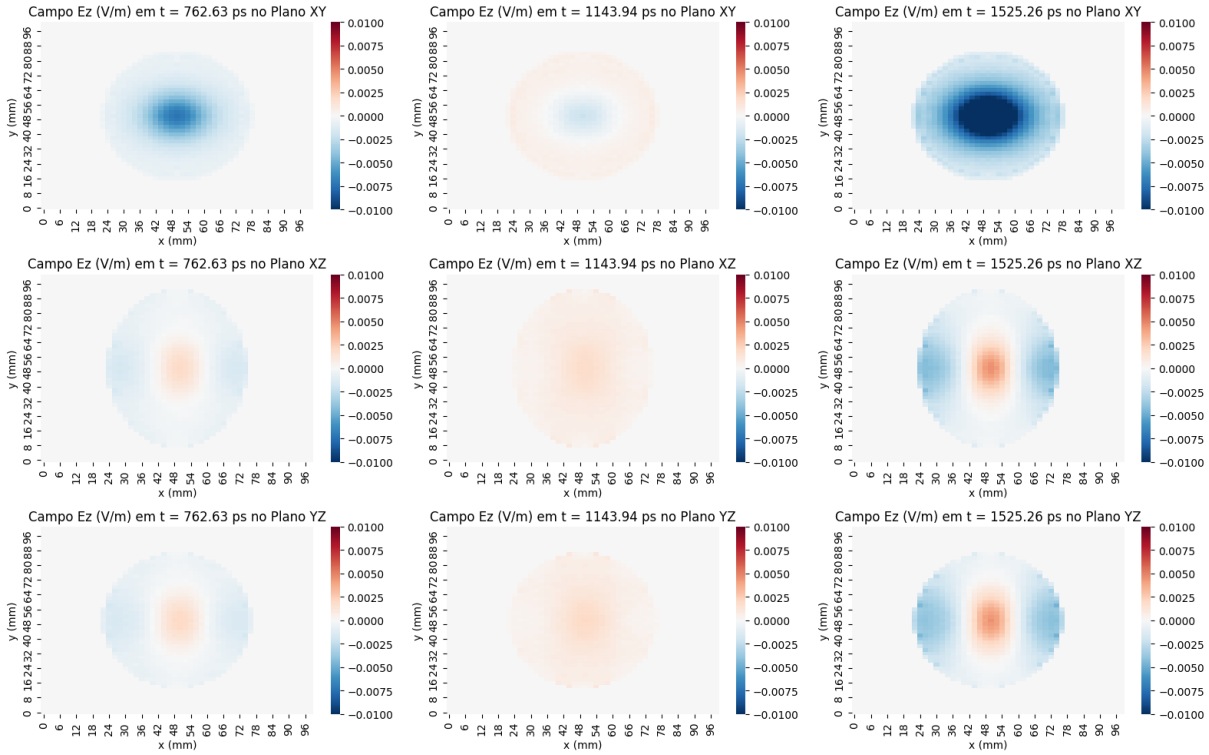
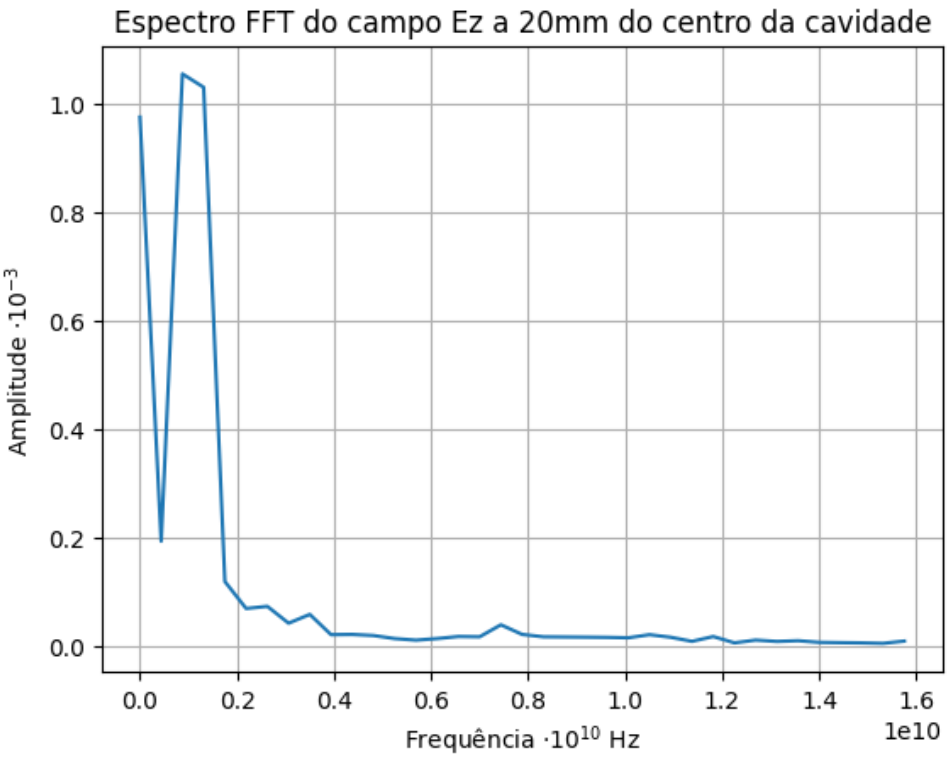


Figura 16 – Transformada de Fourier do sinal coletado no ponto de análise a 20 mm do centro da cavidade.

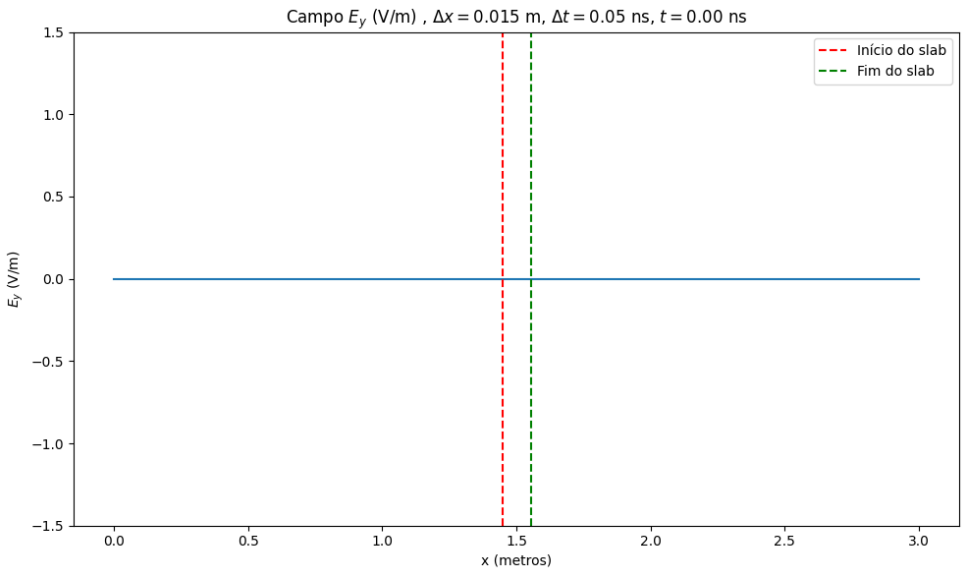


5 EXERCÍCIO 4.3 (LIVRO)

Todo o código usado nesse exercício encontra-se no Anexo D.

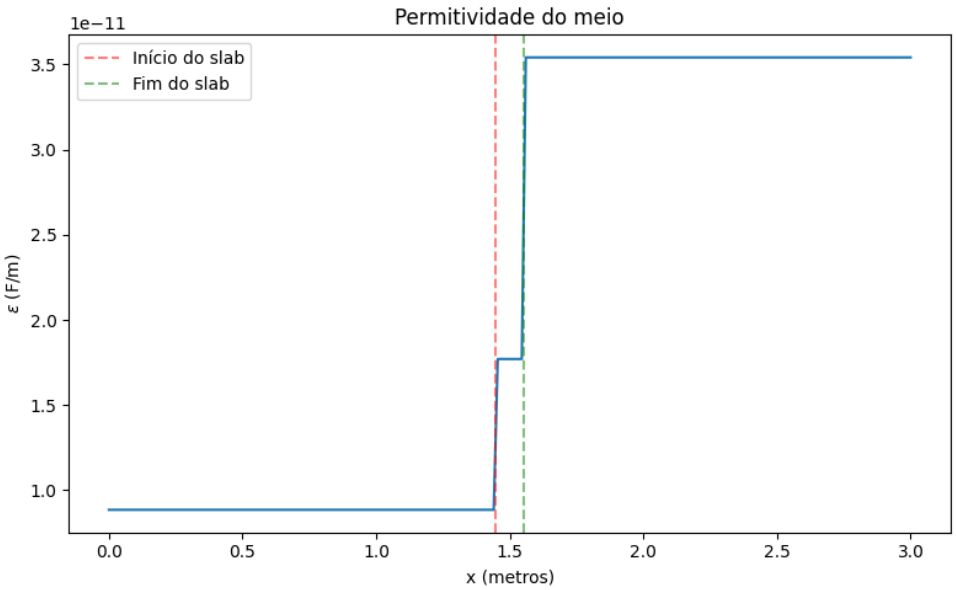
Primeira coisa é desenhar a geometria do problema, como mostra a Figura 17. Temos uma linha de comprimento $L = 3$ m que passa por três tipos de materiais: ar livre, com $\epsilon_1 = \epsilon_0$, um slab de material dielétrico com $\epsilon_2 = 2\epsilon_0$ e depois um segundo dielétrico com $\epsilon_3 = 4\epsilon_0$. μ é constante nos três materiais, sendo igual a μ_0.

Figura 17 – Geometria do exercício 4.3.



A variação da permissividade ao longo da linha é exibida na Figura 18. Vemos que a permissividade é ϵ_0 para $0 \leq x < 1$ m,

Figura 18 – Variação da permissividade ao longo da linha do exercício 4.3.



Os parâmetros de discretização usados foram:

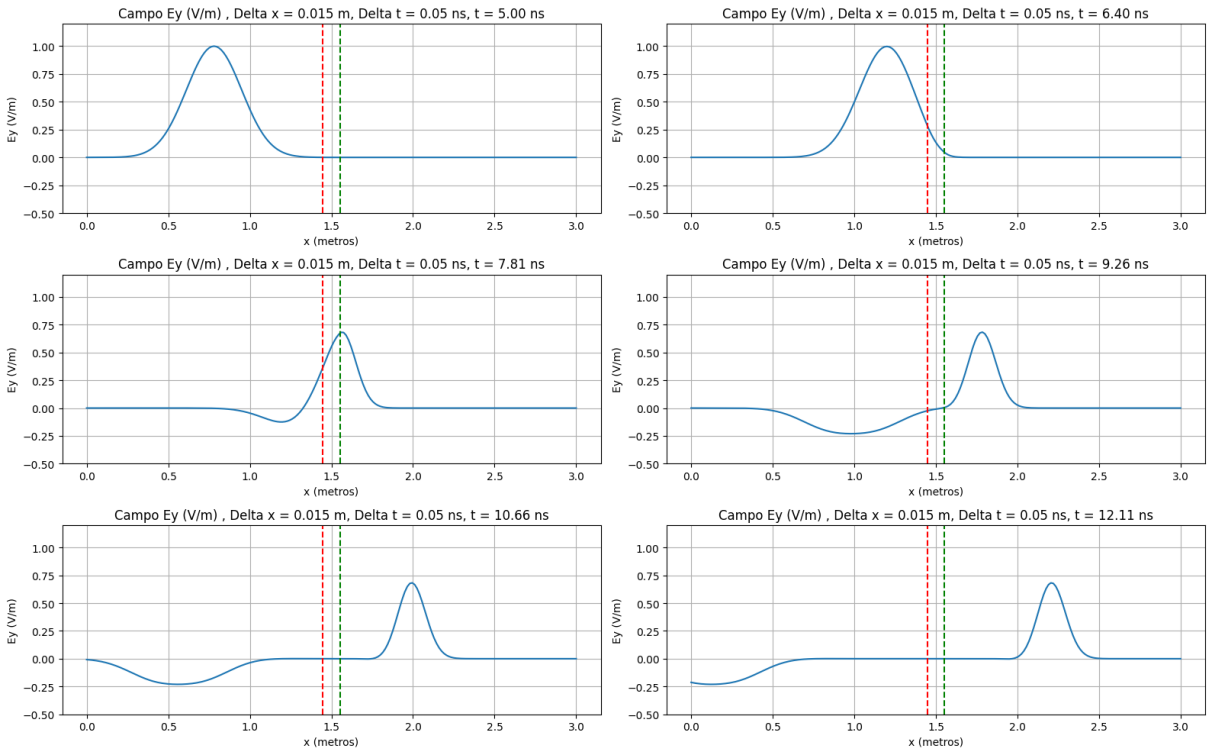
- $N_x = 200 \quad , \quad N_t = 600$
- $\Delta x = 15$ mm
- $\Delta t = 50 \cdot 10^{-12}$ s
- $T = 30$ ns

O campo elétrico é excitado por um pulso gaussiano no início da linha, e o objetivo é observar a propagação do pulso ao longo da linha, incluindo as reflexões e transmissões nos limites entre os materiais. Após a excitação, as condições de contorno nas extremidades são consideradas de um meio infinito, o que implica que as ondas que atingem as extremidades do grid e não sofrem reflexão.

5.1 Item (a)

Usando o esquema de Yee em uma dimensão, obtemos o resultado exibido na Figura 19. Vemos que o pulso gaussiano se propaga para a direita, sendo parcialmente refletido e transmitido em cada interface entre os materiais.

Figura 19 – Evolução do pulso gaussiano ao longo da linha do exercício 4.3.



Vemos na Figura 19 que o sinal incidente tem amplitude de 1 V, enquanto o sinal refletido tem amplitude de 0.25 V, o que resulta em um coeficiente de reflexão de

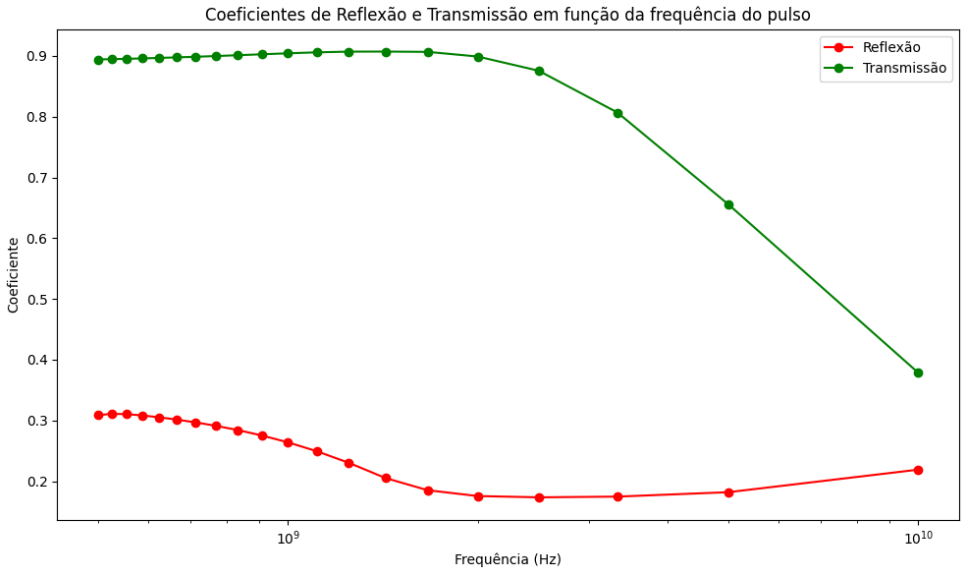
$$R = \frac{0.25}{1} = 0.25$$

Além disso, o sinal transmitido tem amplitude de 0.70 V, o que resulta em um coeficiente de transmissão de

$$T = \frac{0.70}{1} = 0.70$$

Variando a duração $\tau\Delta t$ no pulso gaussiano, obtemos os seguintes coeficientes de reflexão e transmissão para cada valor de frequência $1/(\tau\Delta t)$, exibidos na Figura 20.

Figura 20 – Valores dos coeficientes de reflexão e transmissão do exercício 4.3.



5.2 *Item (b)*

Agora consideramos um slab com metade de dimensão do item (a). A evolução do campo elétrico está exibida na Figura 21, e variação dos coeficientes de reflexão e transmissão está exibida na Figura 22.

Figura 21 – Evolução do pulso gaussiano ao longo da linha do exercício 4.3, com slab de metade da dimensão.

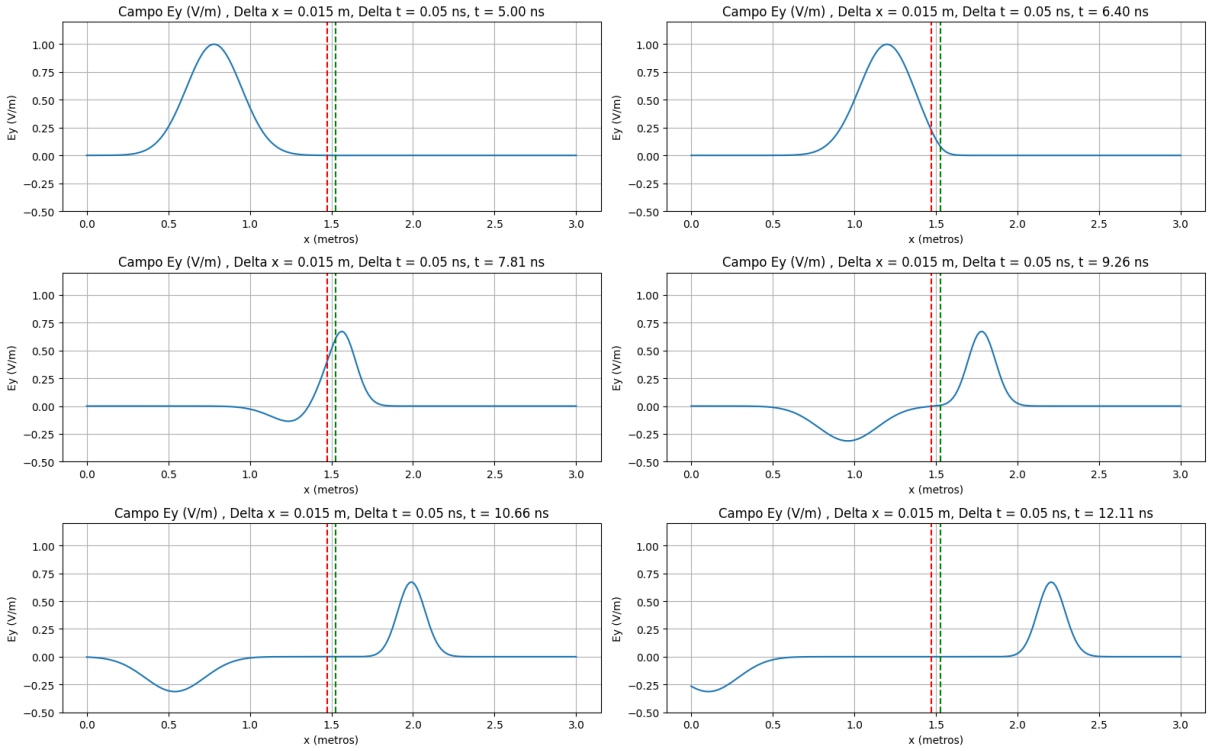
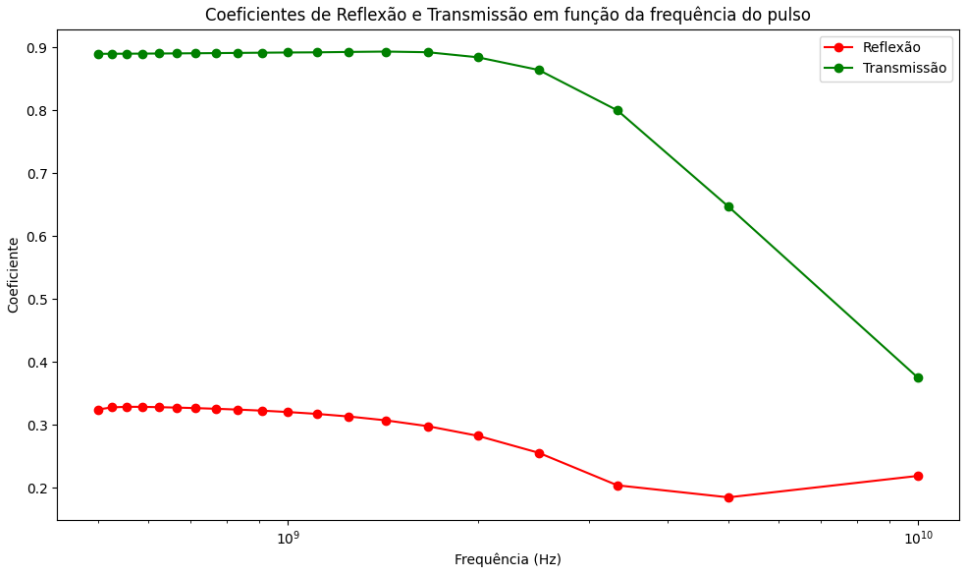


Figura 22 – Valores dos coeficientes de reflexão e transmissão do exercício 4.3, com slab de metade da dimensão.



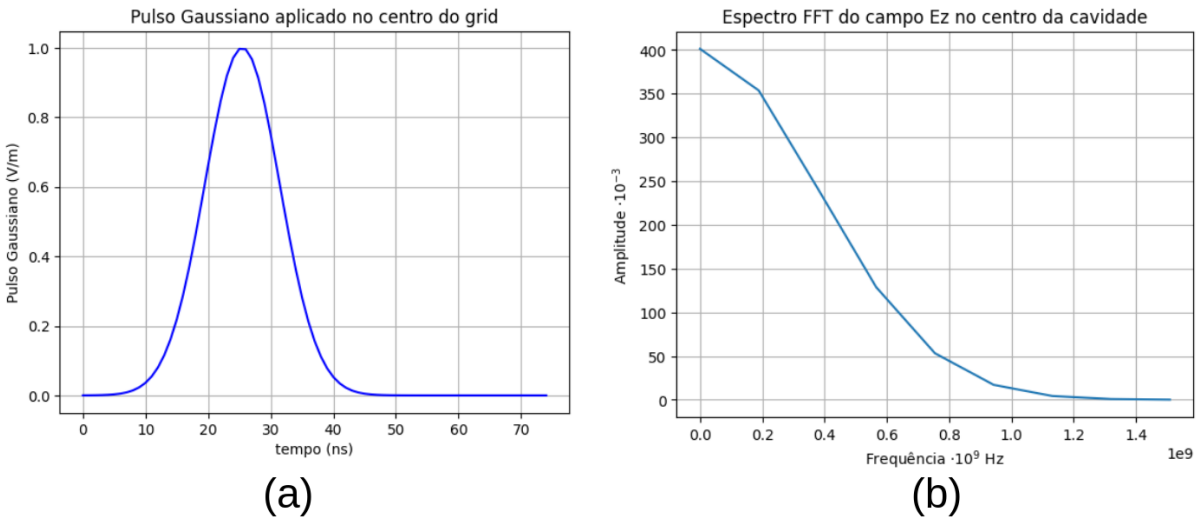
6 EXERCÍCIO 4.4 (LIVRO)

6.1 Item (a)

Todo o código usado nesse exercício encontra-se no Anexo E.

Como a discretização proposta pelo exercício depende do valor de λ_{min}, o primeiro passo é identificar esse valor. O pulso gaussiano que iremos excitar no centro do grid possui a forma exibida na Figura 23 (a), e tem um espectro de frequência exibido na Figura 23 (b).

Figura 23 – Pulso gaussiano (a) e seu espectro de frequência (b).



Como vemos na Figura 23 (b), o espectro se estende até $f_{max} = 1$ GHz, o que implica um comprimento de onda mínimo de

$$\lambda_{min} = \frac{c}{f_{max}} = 0.3 \text{ m}$$

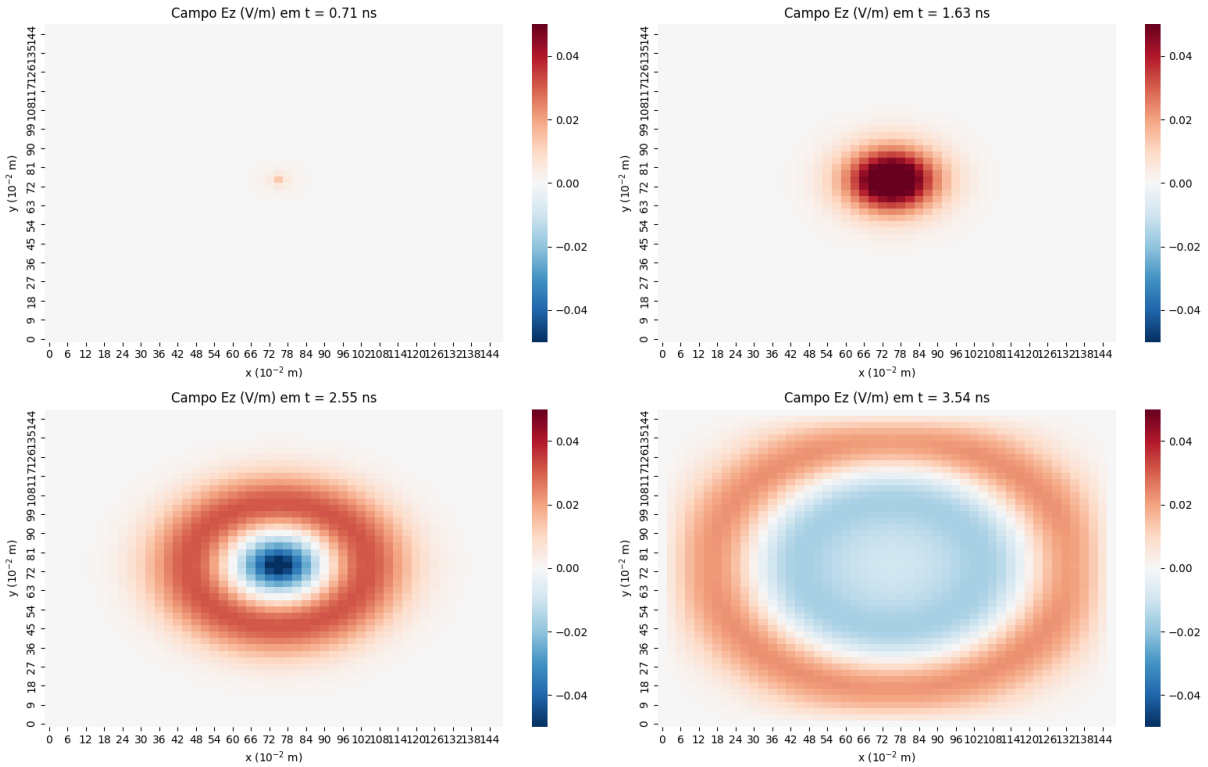
Com esse valor determinado, temos os seguintes parâmetros de discretização:

- $N_x = N_y = 50$
- $\Delta x = \Delta y = 0.1\lambda_{min} = 0.03$ m
- $\Delta t = \frac{\Delta x}{v_p\sqrt{2}} \approx 2.12 \cdot 10^{-11}$ s
- $N_t = 300$

e a evolução do campo elétrico E_z ao longo do tempo está exibida na Figura 24. Note que a figura está “borrada” uma vez que N_x e N_y exigidos pelo enunciado são pequenos.

A evolução do campo magnético H_x ao longo do tempo está exibida na Figura 25. Vemos que o campo magnético é gerado a partir do campo elétrico, e tem um perfil de propagação semelhante.

Figura 24 – Evolução do campo elétrico E_z ao longo do grid.



Como a condição de contorno possui o campo elétrico zerado nas bordas do grid, o pulso gaussiano é refletido nas bordas e cria um padrão de ressonância ao longo do tempo, com interferências destrutivas e construtivas, como mostra a Figura 26.

6.2 *Item (b)*

Agora consideramos uma fonte senoidal no centro do grid, com frequência de $f = 30$ GHz e amplitude unitária. A evolução do campo elétrico E_z ao longo do tempo está exibida na Figura 27.

Vemos que a amplitude do campo é maior no centro do grid, onde a fonte senoidal é aplicada, e decresce à medida que se propaga para fora. Isso está exibido em mais detalhes nas Figura 28. Esse comportamento é conforme o esperado, uma vez que as amplitudes das componetes de E e H decaem com a distância r da fonte com um fator $1/r$ em duas dimensões.

6.3 *Item (c)*

O código usado nesse exercício encontra-se no Anexo F.

Usando o modo Transversal Elétrico (TE) com a excitação gaussiana, obtemos a evolução do campo H_z exibida na Figura 29.

Figura 25 – Evolução do campo magnético H_x ao longo do grid.

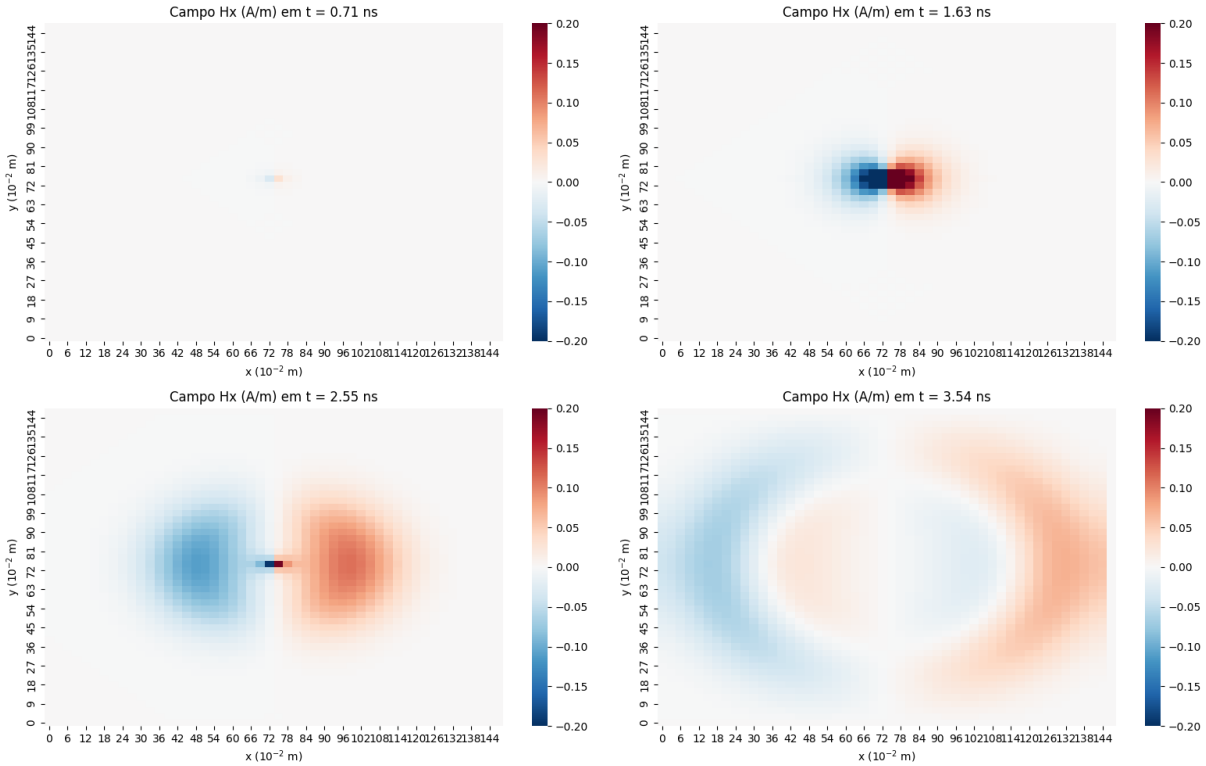


Figura 26 – Padrão de ressonância do pulso gaussiano ao longo do tempo após atingir as bordas do grid.

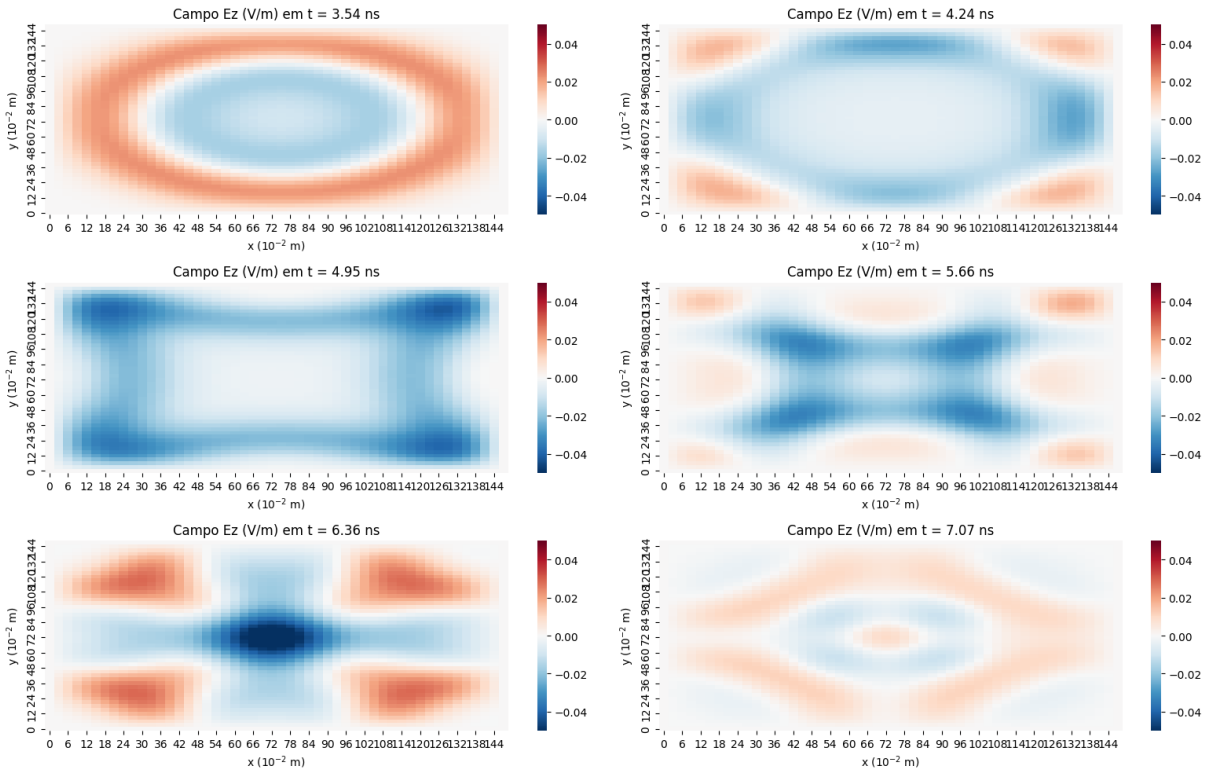


Figura 27 – Evolução do campo elétrico E_z ao longo do grid para uma fonte senoidal.

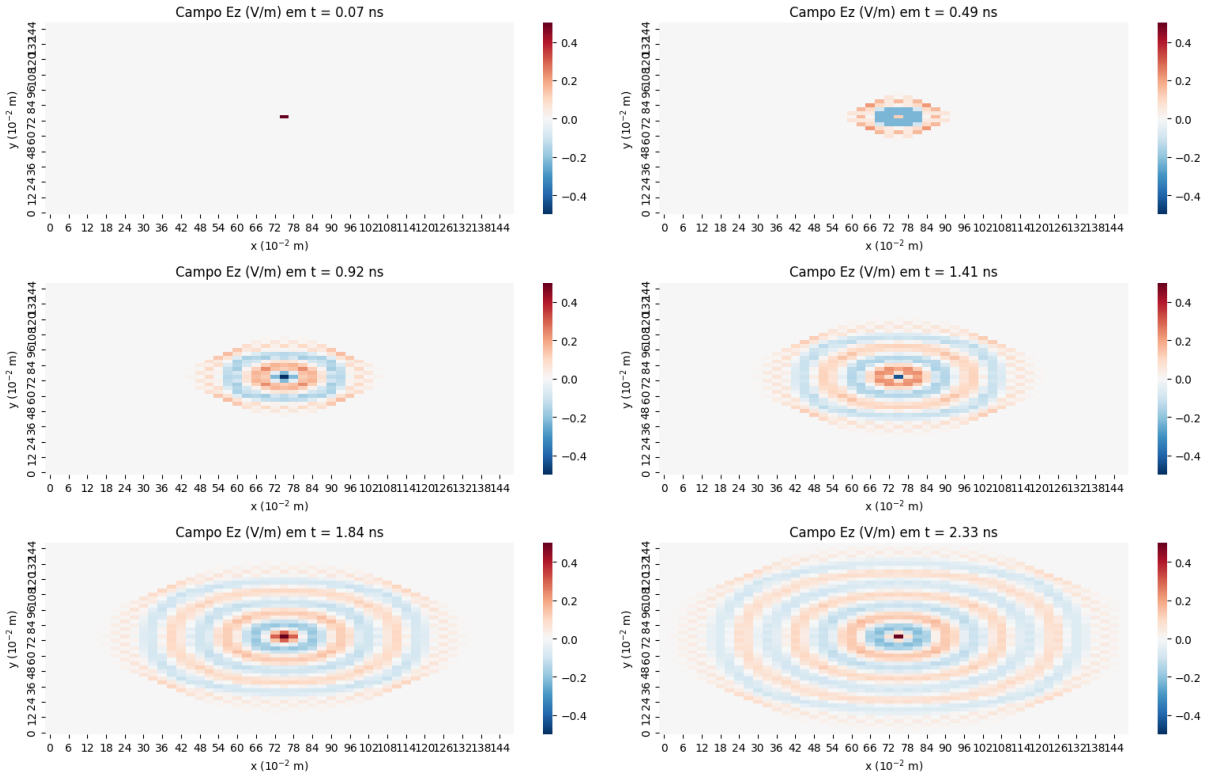


Figura 28 – Decaimento da amplitude de E_z ao se distanciar da fonte.

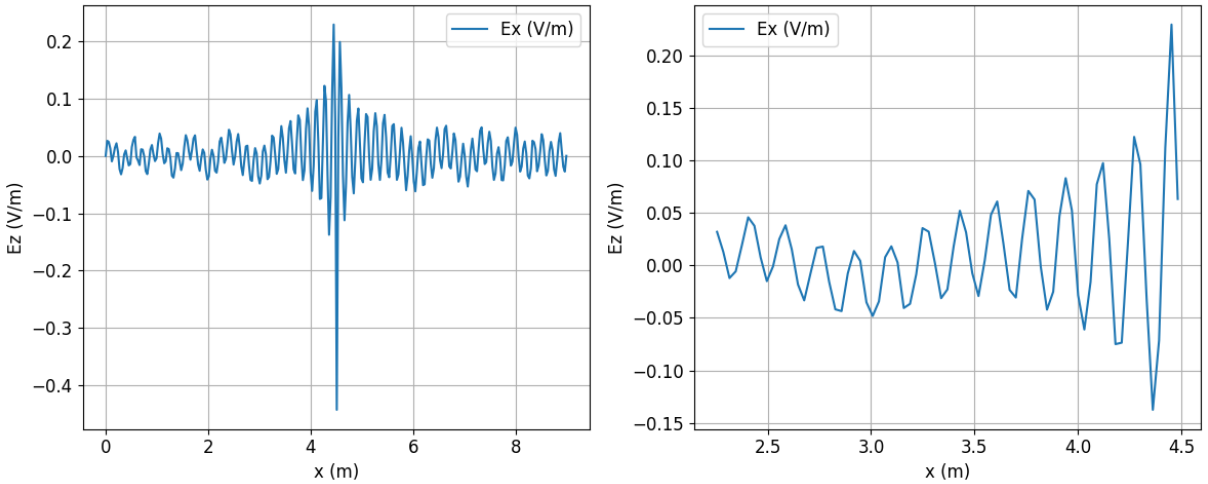
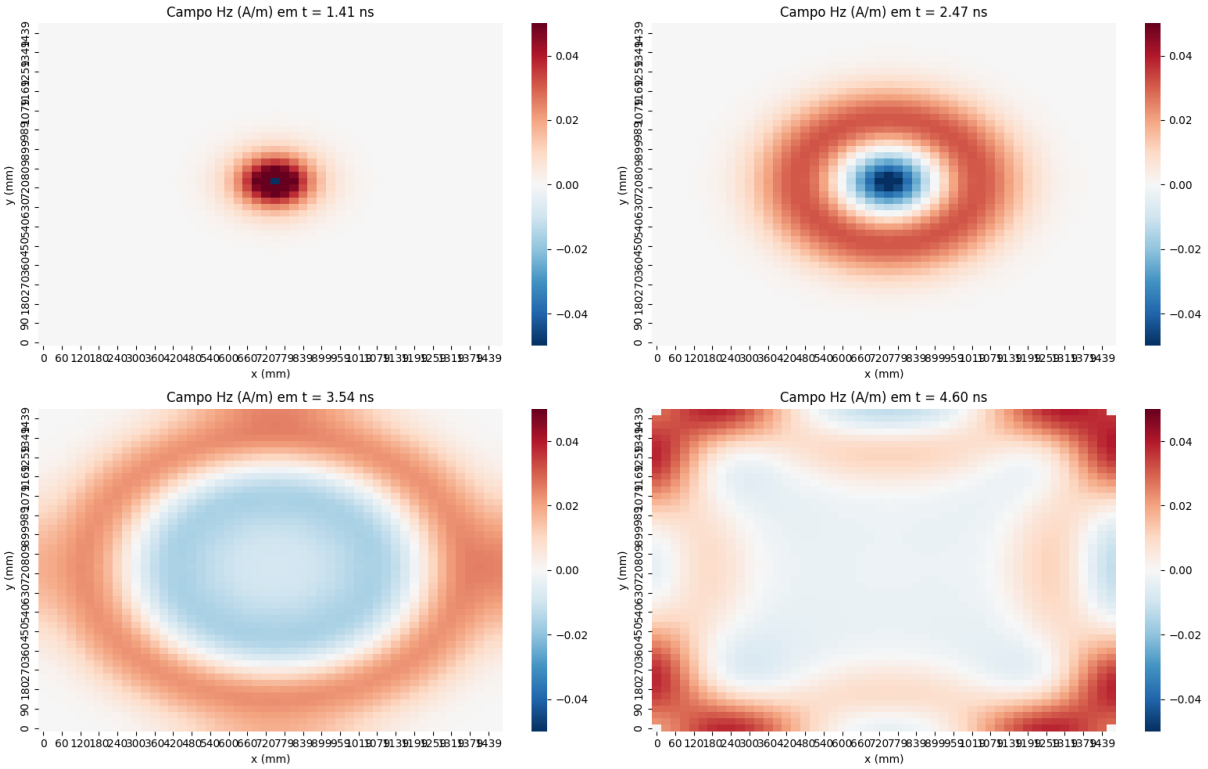


Figura 29 – Evolução do campo H_z para uma excitação gaussiana no modo TE.



7 EXERCÍCIO 4.6 (LIVRO)

Todo o código usado nesse item encontra-se no Anexo G.

7.1 Item (a)

Considerando espaço livre, vamos excitar a antena com uma tensão senoidal dada por

$$V(n) = \sin(2\pi f n \Delta t)$$

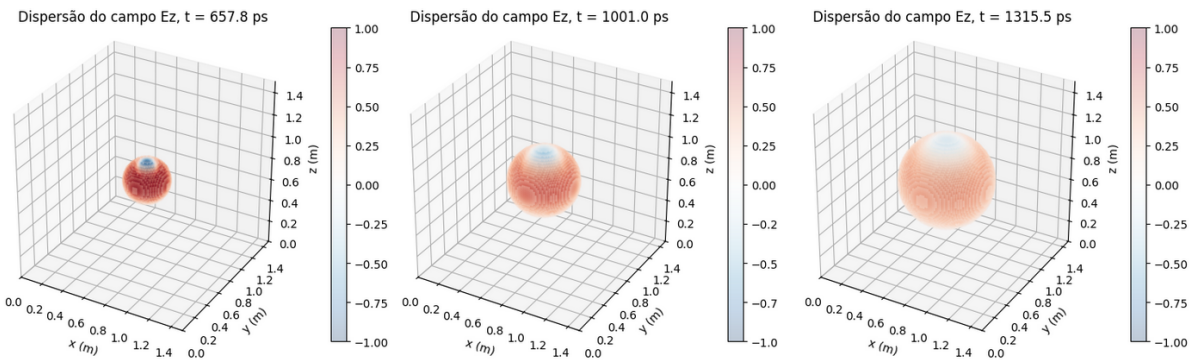
onde $f = 1$ GHz. Assim, o campo gerado $E_z = V(t)/\Delta z$ terá ondas de comprimento de onda

$$\lambda = v/f \quad \Longrightarrow \quad \lambda = 0.3 \text{ m}$$

Assim, para usar 20 células por λ, temos $\Delta x = 15$ mm. Além disso, para evitar que o simulação sofra interferência por reflexão nas bordas, vamos parar a simulação antes de o sinal chegar na parede. Com $N_x = 100$, o sinal atinge as bordas após aproximadamente 2.5 ns. Usando $\Delta t = 28.6$ ps (isso vem da condição CFL 3D), usamos $N_t = 70$ para que a simulação se encerre em aproximadamente 2 ns.

A propagação de E_z a partir da antena está exibido na Figura 30.

Figura 30 – Evolução do campo E_z em um scatter 3D ao longo do tempo.



A Figura 31 mostra o perfil do campo E_z através de 3 planos ortogonais a $10 \cdot \Delta x$ de distância da antena, de modo semelhante ao feito na Questão 4.

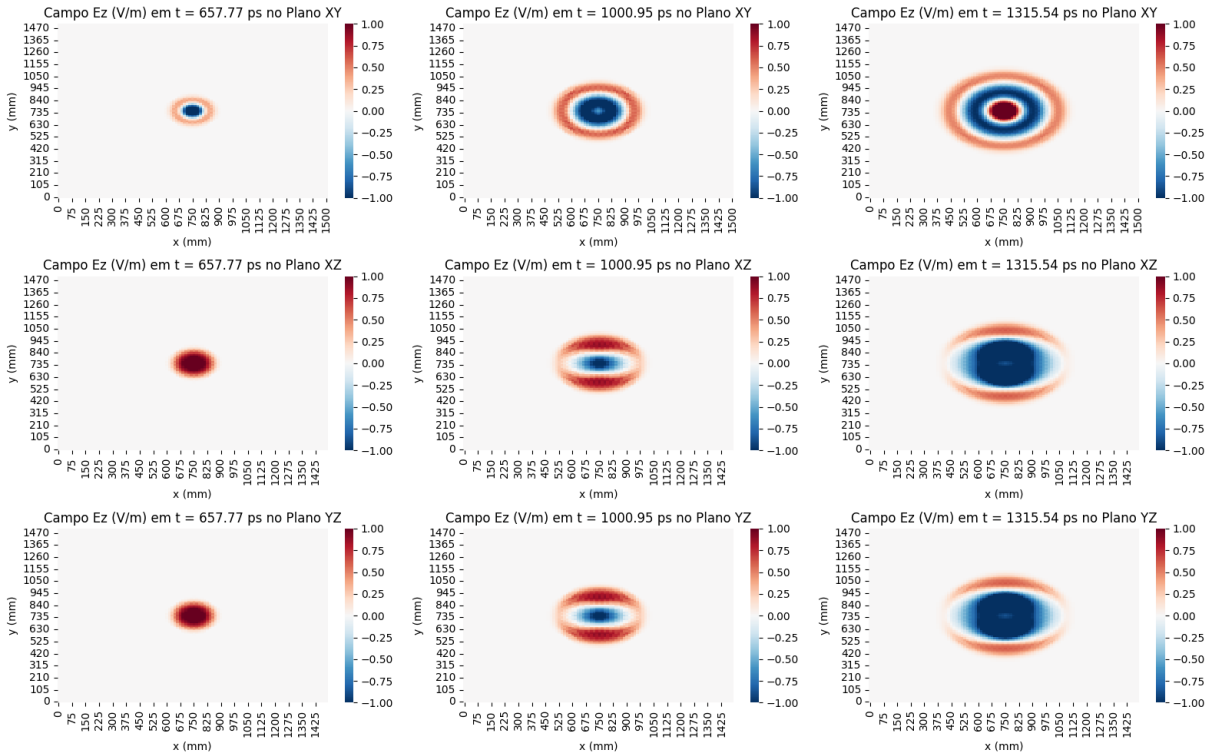
O ponto de monitoramento está a $10 \cdot \Delta x$ da antena nos três eixos e possui as seguintes medições ao longo do tempo, exibidas na Figura 32.

Note que o campo gastou cerca de 800 ps para atingir o ponto de teste. Isso implica que o ponto de teste está a

$$d = v \cdot 800 \text{ ps} \quad \Longrightarrow \quad d = 240 \text{ mm}$$

o que é condizente com a distância $D_{teorica}$ do ponto de teste ao centro da antena:

Figura 31 – Evolução do campo E_z através dos planos ortogonais.

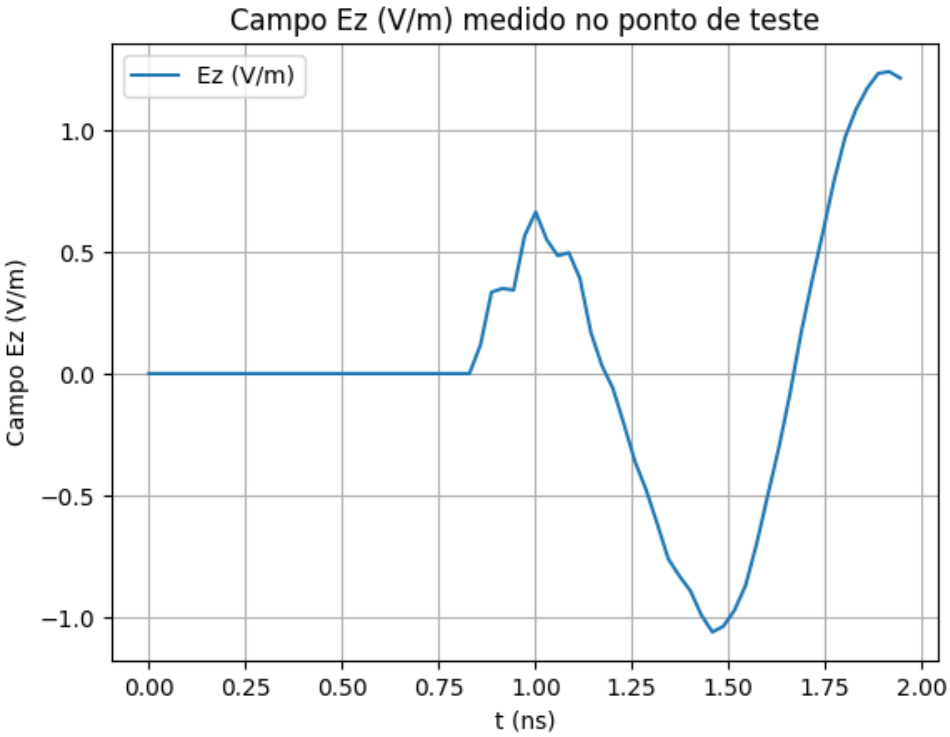


$$D_{teorica} = \sqrt{(10 \cdot \Delta x)^2 + (10 \cdot \Delta y)^2 + (10 \cdot \Delta z)^2} = \sqrt{300(\Delta x)^2} = \sqrt{300}\Delta x = 260 \text{ mm}$$

sendo essa uma nova métrica do bom funcionamento do método codificado.

7.2 Item (b)

Figura 32 – E_z em função do tempo medido no ponto de teste.



ANEXO A – CÓDIGO EXERCÍCIO 2

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams.update({'font.size': 12})

# Constantes
eps = 1
mu = 1
vp = 1 / np.sqrt(eps * mu)

a = 1 # tamanho da reta (eixo x)
T = 1.5 # tempo de simulação

kx = np.pi / a
omega = vp * kx

nx_list = [50, 100, 200, 400]
rel_err_by_nx = []

for nx in nx_list:
    dx = a / nx
    dt = dx / 2
    nt = int(T / dt)

    # grids: armazenam os nós
    # OBS: nx + 1 nós implicam em nx intervalos
    grid_xE = np.linspace(0, a, nx + 1) # xgrid para o Ey ; +1 pois sao nx intervalos de
        delta x
    grid_tE = np.linspace(0, T, nt + 1) # tgrid para o Ey ; +1 pois sao nt intervalos de
        delta t

    grid_xH = np.linspace(dx / 2, a - dx / 2, nx) # xgrid para o Hz ; temos nx - 1
        intervalos de delta x
    grid_tH = np.linspace(dt / 2, T - dt / 2, nt) # tgrid para o Hz ; temos nt - 1
        intervalos de delta t

    E_curr = np.sin(kx * grid_xE) # condicao inicial
    E_next = np.zeros(grid_xE.shape[0])

    H_curr = np.sin(omega * dt/2) * np.cos(kx * grid_xH) # estende a condição inicial para
         o H
    # H_curr = - (dt / (mu0 * dx)) * (E_curr[1:] - E_curr[:-1])
    H_next = np.zeros(grid_xH.shape[0])

    # snapshots para as fotos
    snapshot_times = np.linspace(0, nt - 1, 6, dtype=int)
    E_snapshots = []
    H_snapshots = []
    E_exact_snapshots = []
    H_exact_snapshots = []

    # marcha no tempo - calculamos o next a cada interação
    error_by_n = []
    for n in range(nt):
        # equações do FDTD 1D
        H_next = H_curr - dt / (mu * dx) * (E_curr[1:] - E_curr[:-1])
        E_next[1:-1] = E_curr[1:-1] - dt / (eps * dx) * (H_next[1:] - H_next[:-1])
```

```
    # condição de contorno: PEC nas bordas
    E_next[0] = E_next[-1] = 0

    # Solução exata nos instantes do next, que foram calculados
    E_exact = np.cos(omega * grid_tE[n+1]) * np.sin(kx * grid_xE)
    H_exact = -np.sin(omega * grid_tH[n]) * np.cos(kx * grid_xH)

    # energia da solução exata
    wt = np.sqrt(np.sum((E_exact[:-1]**2 + H_exact**2) * dx))

    # Erro nas normas discretas
    err_E = np.sqrt(dx * np.sum((E_next - E_exact)**2))
    err_H = np.sqrt(dx * np.sum((H_next - H_exact)**2))
    err_total = np.sqrt(err_E**2 + err_H**2)
    error_by_n.append(err_total / wt)

    # tira as fotos
    if n in snapshot_times:
        E_snapshots.append(E_next.copy())
        H_snapshots.append(H_next.copy())
        E_exact_snapshots.append(E_exact.copy())
        H_exact_snapshots.append(H_exact.copy())

    # atualiza o corrente para o proximo passo
    E_curr = E_next
    H_curr = H_next

rel_err_by_nx.append(np.max(error_by_n))
```

ANEXO B – CÓDIGO EXERCÍCIO 3

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
c = 1 / np.sqrt(eps * mu)

# --- domínio ---
a = 0.5
Nx = Ny = 300
dx = dy = 5e-3
cx, cy = Nx//2, Ny//2

dt = 10e-12
Nt = 2000

# --- campos ---
Ez = np.zeros((Nt, Nx, Ny))
Hx = np.zeros((Nt, Nx, Ny))
Hy = np.zeros((Nt, Nx, Ny))

# mascara para a cavidade
x = np.arange(Nx) * dx
y = np.arange(Ny) * dy
X, Y = np.meshgrid(x, y, indexing='ij')

# circle parameters
R = a                # radius (meters)

# boolean mask: True inside the circle
mask = (X - cx * dx)**2 + (Y - cy * dy)**2 <= R**2

# ponta de prova para coletar pontos para a FFT no centro da cavidade
probe = []

for n in range(Nt-1):
    # --- fonte (pulso gaussiano no tempo) ---
    t0 = 50
    spread = 15
    source_gaussiano = np.exp(-((n - t0)**2) / spread**2)

    # --- fonte (senoidal no tempo) ---
    frequency = 1e9  # 1 GHz
    source_senoidal = np.sin(2 * np.pi * frequency * n * dt)

    Ez[n, cx, cy] += source_senoidal

    Hx[n+1, :, :-1] = Hx[n, :, :-1] - (dt/(mu*dy)) * (
        Ez[n, :, 1:] - Ez[n, :, :-1]
    )

    Hy[n+1, :-1, :] = Hy[n, :-1, :] + (dt/(mu*dx)) * (
        Ez[n, 1:, :] - Ez[n, :-1, :]
    )
```

```
    # --- Update E field (from H at time n+1) ---
    Ez[n+1, 1:-1, 1:-1] = Ez[n, 1:-1, 1:-1] + (dt/eps) * (
        (Hy[n+1, 1:-1, 1:-1] - Hy[n+1, :-2, 1:-1]) / dx -
        (Hx[n+1, 1:-1, 1:-1] - Hx[n+1, 1:-1, :-2]) / dy
    )

    # zero nas extremidades e para fora da cavidade (condição de contorno)
    Ez[n+1][~mask] = 0

    # salva o valor no centro para a FFT
    probe.append(Ez[n+1, cx, cy])

# 2. Compute the FFT
yf = fft(probe)
xf = fftfreq(len(probe), dt)

# 3. Plot the results (Positive frequencies only)
plt.plot(xf[:len(probe)//8], 1e3 * 2.0/len(probe) * np.abs(yf[:len(probe)//8]))
plt.title("Espectro FFT do campo Ez no centro da cavidade")
plt.xlabel(r"Frequência $\cdot 10^{10}$ Hz")
plt.ylabel(r"Amplitude $\cdot 10^{-3}$")
plt.grid()
plt.show()
```

ANEXO C – CÓDIGO EXERCÍCIO 4

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
a = 0.05
b = 0.04
c = 0.03

Nx = Ny = Nz = 50
dx = dy = dz = 2e-3
cx, cy, cz = Nx // 2, Ny // 2, Nz // 2
L = Nx * dx # dimensao do grid

# condição CFL para estabilidade
dt = 0.99 / (vp * np.sqrt(
    (1/dx**2) + (1/dy**2) + (1/dz**2)
))
Nt = 600
T = Nt * dt

# define os grids - desenha a célula de Yee 3D
grid_x_base = np.linspace(0, L, Nx + 1)
grid_y_base = np.linspace(0, L, Ny + 1)
grid_z_base = np.linspace(0, L, Nz + 1)

grid_x_Ex = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Ex = np.linspace(0, L, Ny + 1)
grid_z_Ex = np.linspace(0, L, Nz + 1)

grid_x_Ey = np.linspace(0, L, Nx + 1)
grid_y_Ey = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Ey = np.linspace(0, L, Nz + 1)

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_z_Ez = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hx = np.linspace(0, L, Nx + 1)
grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Hx = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hy = np.linspace(0, L, Ny + 1)
grid_z_Hy = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hz = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hz = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Hz = np.linspace(0, L, Nz + 1)

# ponta de prova para coletar pontos para a FFT no centro da cavidade
```

```
probe = []
pcx, pcy, pcz = cx + 10, cy + 10, cz + 10

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(10, Nt - 2, 3, dtype=int)
# n_snapshots = range(Nt-1)
n_snapshots = [Nt // 3, Nt//2, 2 * (Nt // 3)]
Ez_snapshots = []

Ex = Ex_next = np.zeros((grid_x_Ex.shape[0], grid_y_Ex.shape[0],  grid_z_Ex.shape[0]))
Ey = Ey_next = np.zeros((grid_x_Ey.shape[0], grid_y_Ey.shape[0],  grid_z_Ey.shape[0]))
Ez = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0],  grid_z_Ez.shape[0]))
Hx = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0],  grid_z_Hx.shape[0]))
Hy = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0],  grid_z_Hy.shape[0]))
Hz = Hz_next = np.zeros((grid_x_Hz.shape[0], grid_y_Hz.shape[0],  grid_z_Hz.shape[0]))

X, Y, Z = np.meshgrid(grid_x_Ez, grid_y_Ez, grid_z_Ez, indexing='ij')
mask = (X - cx * dx)**2 / a**2 + (Y - cy * dy)**2 / b**2 + (Z - cz * dz)**2 / c**2 <= 1

for n in range(Nt-1):
    # --- fonte (pulso gaussiano no tempo) ---
    t0 = 50
    spread = 15
    source_gaussiano = np.exp(-((n - t0)**2) / spread**2)

    # --- fonte (senoidal no tempo) ---
    frequency = 1e9  # 1 GHz
    source_senoidal = np.sin(2 * np.pi * frequency * n * dt)

    Ez[cx, cy, cz] += source_senoidal

    # --- Update H fields ---
    # Hx
    Hx_next[:, :, :] = Hx[:, :, :] - (dt/mu)*(
        (Ez[:, 1:, :] - Ez[:, :-1, :])/dy -
        (Ey[:, :, 1:] - Ey[:, :, :-1])/dz
    )

    # Hy
    Hy_next[:, :, :] = Hy[:, :, :] - (dt/mu)*(
        (Ex[:, :, 1:] - Ex[:, :, :-1])/dz -
        (Ez[1:, :, :] - Ez[:-1, :, :])/dx
    )

    # Hz
    Hz_next[:, :, :] = Hz[:, :, :] - (dt/mu)*(
        (Ey[1:, :, :] - Ey[:-1, :, :])/dx -
        (Ex[:, 1:, :] - Ex[:, :-1, :])/dy
    )

    # --- Update E fields ---
    # Ex
    Ex_next[:, 1:-1, 1:-1] = Ex[:, 1:-1, 1:-1] + (dt/eps)*(
        (Hz_next[:, 1:, 1:-1] - Hz_next[:, :-1, 1:-1])/dy -
        (Hy_next[:, 1:-1, 1:] - Hy_next[:, 1:-1, :-1])/dz
    )

    # Ey
    Ey_next[1:-1, :, 1:-1] = Ey[1:-1, :, 1:-1] + (dt/eps)*(
        (Hx_next[1:-1, :, 1:] - Hx_next[1:-1, :, :-1])/dz -
```

```
    (Hz_next[1:, :, 1:-1] - Hz_next[:-1, :, 1:-1])/dx
)

# Ez
Ez_next[1:-1, 1:-1, :] = Ez[1:-1, 1:-1, :] + (dt/eps)*(
    (Hy_next[1:, 1:-1, :] - Hy_next[:-1, 1:-1, :])/dx -
    (Hx_next[1:-1, 1:, :] - Hx_next[1:-1, :-1, :])/dy
)

# zero nas extremidades e para fora da cavidade (condição de contorno)
Ez_next[~mask] = 0

# salva o valor no centro para a FFT
probe.append(Ez_next[pcx, pcy, pcz])

# atualiza os campos para o próximo passo
Ex, Ex_next = Ex_next, Ex
Ey, Ey_next = Ey_next, Ey
Ez, Ez_next = Ez_next, Ez

Hx, Hx_next = Hx_next, Hx
Hy, Hy_next = Hy_next, Hy
Hz, Hz_next = Hz_next, Hz

# salva para os snapshots
if n in n_snapshots:
    Ez_snapshots.append(Ez.copy())
```

ANEXO D – CÓDIGO EXERCÍCIO 4.3

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Parameters
e0 = 8.854187817e-12  # Vacuum permittivity (F/m)
mu0 = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(e0 * mu0)

# free space
e1 = 1 * e0
mu1 = 1 * mu0

# dieletric slab
e2 = 2 * e0
mu2 = 1 * mu0

# second dielectric to the right
e3 = 4 * e0
mu3 = 1 * mu0

lambda0 = 0.3
dx = lambda0 / (20 * np.sqrt(e1 / e0))
dt = dx / vp

L = 3
Nx = int(L / dx) + 1
Nt = 600
T = Nt * dt

# tau = 0.8e-9 / dt
# tau = 0.2e-9 / dt

tau_list = np.linspace(0.1, 2, 20) * 1e-9 / dt
# tau_list = np.array([0.8e-9 / dt])
r_list = []
transmit_list = []

for tau in tau_list:
    # inicio e fim do slab
    d = lambda0 / (4 * np.sqrt(e2 / e0)) # tamanho do slab
    center = Nx // 2
    center_m = center * dx
    start_d = center_m - d / 2
    end_d = center_m + d / 2

    def gaussian(n):
        return np.exp(-(n - 3 * tau)**2 / tau**2)

    # campos
    Ey = np.zeros((Nt, Nx))
    Hz = np.zeros((Nt, Nx))

    # arrays para as propriedades dos materiais
    e_medium = np.zeros(Nx)
    mu_medium = np.zeros(Nx)

    for i in range(Nx):
```

```
    xcoord = i * dx
    if xcoord < start_d: # ar livre
        e_medium[i] = e1
        mu_medium[i] = mu1
    elif start_d <= xcoord <= end_d: # slab
        e_medium[i] = e2
        mu_medium[i] = mu2
    elif xcoord > end_d: # segundo dieletrico
        e_medium[i] = e3
        mu_medium[i] = mu3

# marcha no tempo
for n in range(Nt - 1):

    # atualiza o H
    for i in range(Nx - 1):
        Hz[n+1, i] = Hz[n, i] + (dt / (mu_medium[i] * dx)) * (Ey[n, i+1] - Ey[n, i])

    # atualiza o E
    for i in range(1, Nx - 1):
        # por causa do i-1 no final que o range começa em 1
        Ey[n+1, i] = Ey[n, i] + (dt / (e_medium[i] * dx)) * (Hz[n+1, i] - Hz[n+1, i
            -1])

    # condição de contorno
    if n < 6 * tau:  # só aplica a fonte nos primeiros instantes
        Ey[n+1, 0] = gaussian(n+1) # esquerda
    else: # condicao de contorno absorvente
        Ey[n+1, 0] = Ey[n, 1] + (vp * dt - dx) / (vp * dt + dx) * (Ey[n+1, 1] - Ey[n,
            0])  # absorvente à esquerda
    Ey[n+1, -1] = Ey[n, -2] - (vp * dt - dx) / (vp * dt + dx) * (Ey[n+1, -2] - Ey[n,
        -1])  # absorvente à direita


# coeficiente de reflexão - minimo valor medido no primeiro interval
R = np.abs(np.min(Ey[:, 0]) / np.max(Ey[:, 0]))
r_list.append(R)

# coeficiente de tranmissão - maximo valor medido no ultimo intervalo
T = np.max(Ey[:, -1]) / 1
transmit_list.append(T)
```

ANEXO E – CÓDIGO EXERCÍCIO 4.4 - MODO TM

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
min_lambda = vp / 1e9  # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = Ny = 50
dx = dy = 0.1 * min_lambda
dt = dx / (vp * np.sqrt(2))
Nt = 200

cx, cy = Nx // 2, Ny // 2

# --- campos ---
Ez = np.zeros((Nt, Nx, Ny))
Hx = np.zeros((Nt, Nx, Ny))
Hy = np.zeros((Nt, Nx, Ny))

for n in range(Nt-1):
    # --- fonte (pulso gaussiano no tempo) ---
    source_gaussiano = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)

    # --- fonte (senoidal no tempo) ---
    frequency = 30e9
    source_senoidal = np.sin(2 * np.pi * frequency * n * dt)

    Ez[n, cx, cy] += source_senoidal

    Hx[n+1, :, :-1] = Hx[n, :, :-1] - (dt/(mu*dy)) * (
        Ez[n, :, 1:] - Ez[n, :, :-1]
    )

    Hy[n+1, :-1, :] = Hy[n, :-1, :] + (dt/(mu*dx)) * (
        Ez[n, 1:, :] - Ez[n, :-1, :]
    )

    # --- Update E field (from H at time n+1) ---
    Ez[n+1, 1:-1, 1:-1] = Ez[n, 1:-1, 1:-1] + (dt/eps) * (
        (Hy[n+1, 1:-1, 1:-1] - Hy[n+1, :-2, 1:-1]) / dx -
        (Hx[n+1, 1:-1, 1:-1] - Hx[n+1, 1:-1, :-2]) / dy
    )

    # zero nas nas extremidades - PEC
    Ez[n+1, 1:-1, 0] = 0
    Ez[n+1, 1:-1, -1] = 0
    Ez[n+1, 0, 1:-1] = 0
    Ez[n+1, -1, 1:-1] = 0
```

ANEXO F – CÓDIGO EXERCÍCIO 4.4 - MODO TE

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

# --- domínio ---
min_lambda = vp / 1e9  # comprimento de onda mínimo para 1 GHz = 3e-1 m
Nx = Ny = 50
dx = dy = 0.1 * min_lambda
dt = dx / (vp * np.sqrt(2))
Nt = 100
L = Nx * dx
T = Nt * dt

cx, cy = Nx // 2, Ny // 2

# modo TE 2D -> sempre olhe a celula de Yee no livro para ver onde cada campo é definido
grid_x_Hz = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hz = np.linspace(dy/2, L - dy/2, Ny)
grid_t_Hz = np.linspace(dt/2, T - dt/2, Nt)

grid_x_Ex = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Ex = np.linspace(0, L, Ny + 1)
grid_t_Ex = np.linspace(0, T, Nt + 1)

grid_x_Ey = np.linspace(0, L, Nx + 1)
grid_y_Ey = np.linspace(dy/2, L - dy/2, Ny)
grid_t_Ey = np.linspace(0, T, Nt + 1)

# --- campos ---
Hz_curr = Hz_next = np.zeros((grid_x_Hz.shape[0], grid_y_Hz.shape[0]))
Ex_curr = Ex_next = np.zeros((grid_x_Ex.shape[0], grid_y_Ex.shape[0]))
Ey_curr = Ey_next = np.zeros((grid_x_Ey.shape[0], grid_y_Ey.shape[0]))

# ponta de prova para coletar pontos para a FFT no centro da cavidade
probe = []

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(20, Nt - 5, 6, dtype=int)
Ez_snapshots = []

for n in range(Nt-1):
    # --- fonte (pulso gaussiano no tempo) ---
    source_gaussiano = np.exp(-((n * dt - 1.8e-9)**2) / (0.6e-9)**2)

    # --- fonte (senoidal no tempo) ---
    frequency = 30e9
    source_senoidal = np.sin(2 * np.pi * frequency * n * dt)

    Hz_curr[cx, cy] += source_gaussiano

    Ex_next[1:-1, 1:-1] = Ex_curr[1:-1, 1:-1] + (dt/(eps*dy)) * (
        Hz_curr[1:-1, 1:] - Hz_curr[1:-1, :-1]
```

```
)

Ey_next[1:-1, 1:-1] = Ey_curr[1:-1, 1:-1] - (dt/(eps*dx)) * (
    Hz_curr[1:, 1:-1] - Hz_curr[:-1, 1:-1]
)

# zero nas nas extremidades - PEC
Ex_next[:, 0] = 0
Ex_next[:, -1] = 0
Ex_next[0, :] = 0
Ex_next[-1, :] = 0

Ey_next[:, 0] = 0
Ey_next[:, -1] = 0
Ey_next[0, :] = 0
Ey_next[-1, :] = 0

Hz_next[:, :] = Hz_curr[:, :] + (dt/mu) * (
    (Ex_next[:, 1:] - Ex_next[:, :-1]) / dx -
    (Ey_next[1:, :] - Ey_next[:-1, :]) / dy
)

# salva o valor no centro para a FFT
probe.append(Hz_next[cx + 5, cy + 5])

# atualiza os campos para o próximo passo
Hz_curr = Hz_next.copy()
Ex_curr = Ex_next.copy()
Ey_curr = Ey_next.copy()

# salva para os snapshots
if n in n_snapshots:
    Ez_snapshots.append(Hz_curr.copy())
```

ANEXO G – CÓDIGO EXERCÍCIO 4.6

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.fft import fft, fftfreq

# --- parâmetros físicos ---
eps = 8.854187817e-12  # Vacuum permittivity (F/m)
mu = 4 * np.pi * 1e-7  # Vacuum permeability (H/m)
vp = 1 / np.sqrt(eps * mu)

Nx = Ny = Nz = 100
dx = dy = dz = 15e-3
cx, cy, cz = Nx // 2, Ny // 2, Nz // 2
L = Nx * dx # dimensao do grid

# condição CFL para estabilidade
dt = 0.99 / (vp * np.sqrt(
    (1/dx**2) + (1/dy**2) + (1/dz**2)
))
Nt = 70
T = Nt * dt

# define os grids - desenha a célula de Yee 3D
grid_x_base = np.linspace(0, L, Nx + 1)
grid_y_base = np.linspace(0, L, Ny + 1)
grid_z_base = np.linspace(0, L, Nz + 1)

grid_x_Ex = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Ex = np.linspace(0, L, Ny + 1)
grid_z_Ex = np.linspace(0, L, Nz + 1)

grid_x_Ey = np.linspace(0, L, Nx + 1)
grid_y_Ey = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Ey = np.linspace(0, L, Nz + 1)

grid_x_Ez = np.linspace(0, L, Nx + 1)
grid_y_Ez = np.linspace(0, L, Ny + 1)
grid_z_Ez = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hx = np.linspace(0, L, Nx + 1)
grid_y_Hx = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Hx = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hy = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hy = np.linspace(0, L, Ny + 1)
grid_z_Hy = np.linspace(dz/2, L - dz/2, Nz)

grid_x_Hz = np.linspace(dx/2, L - dx/2, Nx)
grid_y_Hz = np.linspace(dy/2, L - dy/2, Ny)
grid_z_Hz = np.linspace(0, L, Nz + 1)

# ponta de prova para coletar pontos
probe = []
pcx, pcy, pcz = cx + 10, cy + 10, cz + 10

# valores do tempo para salvar os snapshots e montar os heatmaps
n_snapshots = np.linspace(0, Nt - 2, 3, dtype=int)
# n_snapshots = range(Nt-1)
```

```
n_snapshots = [Nt // 3, Nt//2, 2 * (Nt // 3)]
# n_snapshots = [50, 60, 68]
Ez_snapshots = []

Ex = Ex_next = np.zeros((grid_x_Ex.shape[0], grid_y_Ex.shape[0],  grid_z_Ex.shape[0]))
Ey = Ey_next = np.zeros((grid_x_Ey.shape[0], grid_y_Ey.shape[0],  grid_z_Ey.shape[0]))
Ez = Ez_next = np.zeros((grid_x_Ez.shape[0], grid_y_Ez.shape[0],  grid_z_Ez.shape[0]))
Hx = Hx_next = np.zeros((grid_x_Hx.shape[0], grid_y_Hx.shape[0],  grid_z_Hx.shape[0]))
Hy = Hy_next = np.zeros((grid_x_Hy.shape[0], grid_y_Hy.shape[0],  grid_z_Hy.shape[0]))
Hz = Hz_next = np.zeros((grid_x_Hz.shape[0], grid_y_Hz.shape[0],  grid_z_Hz.shape[0]))

for n in range(Nt-1):
    # --- fonte (senoidal no tempo) ---
    frequency = 1e9  # 1 GHz
    voltage_senoidal = np.sin(2 * np.pi * frequency * n * dt)

    Ez[cx, cy, cz] += voltage_senoidal / dz

    # --- Update H fields ---
    # Hx
    Hx_next[:, :, :] = Hx[:, :, :] - (dt/mu)*(
        (Ez[:, 1:, :] - Ez[:, :-1, :])/dy -
        (Ey[:, :, 1:] - Ey[:, :, :-1])/dz
    )

    # Hy
    Hy_next[:, :, :] = Hy[:, :, :] - (dt/mu)*(
        (Ex[:, :, 1:] - Ex[:, :, :-1])/dz -
        (Ez[1:, :, :] - Ez[:-1, :, :])/dx
    )

    # Hz
    Hz_next[:, :, :] = Hz[:, :, :] - (dt/mu)*(
        (Ey[1:, :, :] - Ey[:-1, :, :])/dx -
        (Ex[:, 1:, :] - Ex[:, :-1, :])/dy
    )

    # --- Update E fields ---
    # Ex
    Ex_next[:, 1:-1, 1:-1] = Ex[:, 1:-1, 1:-1] + (dt/eps)*(
        (Hz_next[:, 1:, 1:-1] - Hz_next[:, :-1, 1:-1])/dy -
        (Hy_next[:, 1:-1, 1:] - Hy_next[:, 1:-1, :-1])/dz
    )

    # Ey
    Ey_next[1:-1, :, 1:-1] = Ey[1:-1, :, 1:-1] + (dt/eps)*(
        (Hx_next[1:-1, :, 1:] - Hx_next[1:-1, :, :-1])/dz -
        (Hz_next[1:, :, 1:-1] - Hz_next[:-1, :, 1:-1])/dx
    )

    # Ez
    Ez_next[1:-1, 1:-1, :] = Ez[1:-1, 1:-1, :] + (dt/eps)*(
        (Hy_next[1:, 1:-1, :] - Hy_next[:-1, 1:-1, :])/dx -
        (Hx_next[1:-1, 1:, :] - Hx_next[1:-1, :-1, :])/dy
    )

    # fixa 0 em 5 celulas de grid acima e abaixo da antena, sendo PEC
    Ez_next[cx, cy, (cz-5):(cz)] = 0
    Ez_next[cx, cy, (cz+1):(cz+6)] = 0
```

```
# salva o valor no centro para a FFT
probe.append(Ez_next[pcx, pcy, pcz])

# atualiza os campos para o próximo passo
Ex, Ex_next = Ex_next, Ex
Ey, Ey_next = Ey_next, Ey
Ez, Ez_next = Ez_next, Ez

Hx, Hx_next = Hx_next, Hx
Hy, Hy_next = Hy_next, Hy
Hz, Hz_next = Hz_next, Hz

# salva para os snapshots
if n in n_snapshots:
    Ez_snapshots.append(Ez.copy())
```